

Implementacja algorytmu mnożenia liczb dowolnej precyzji

Cybulski Mateusz, Derron Tomasz, Domiński Bartosz

Streszczenie

W niniejszej pracy zajęliśmy się implementacją algorytmu Toom-3, który służy do szybkiego mnożenia bardzo dużych liczb. Głównym celem było stworzenie własnej struktury BigInt w języku C++ oraz sprawdzenie, jak nasze rozwiązanie wypada na tle standardów, takich jak biblioteka GMP czy wbudowane mechanizmy języka Python.

W trakcie pracy skupiliśmy się na zaprogramowaniu rozwiązań, które ze sobą współgrają. Opisaliliśmy, jak posłużyliśmy się gotowymi bibliotekami oraz proces twórczy naszego rozwiązania.

Przeprowadzony eksperyment pokazał, że nasza implementacja, choć z teoretyczną złożonością, to wciąż ustępuje profesjonalnym bibliotekom, które są tworzone i optymalizowane od wielu lat.

Spis treści

Słownik pojęć	2
1 Wprowadzenie	3
2 Metodologia	8
3 BigInt	10
4 GMP	12
5 Implementacja w języku Python	12
6 Analiza uzyskanych wyników	12
7 Dodatek	23
7.1 Dokumentacja promptów	23
7.2 Jak duże liczby można pomnożyć w ciągu sekundy?	25
7.3 Dokumentacja techniczna	27

Słownik pojęć

Pojęcie	Opis
Słowo maszynowe (Limb) (słowo)	Podstawowa jednostka danych, na której operuje procesor. W pracy przyjęto słowa 32-bitowe (<code>uint32_t</code>). Biblioteka GMP określa je terminem <i>limb</i> .
Mnożenie szkolne (grade-school multiplication)	Metoda mnożenia, która odpowiada mnożeniu pisemnym. Mnożymy każdą cyfrę z każdą, otrzymując złożoność kwadratową.
Złożoność obliczeniowa	Miara ilości zasobów (czasu lub pamięci) potrzebnych do wykonania algorytmu, wyrażana zazwyczaj w funkcji rozmiaru danych wejściowych n .
Notacja dużego O	Sposób opisu asymptotycznego tempa wzrostu funkcji, pozwalający klasyfikować algorytmy według ich wydajności.
Most Significant Word (MSW)	Najbardziej znaczące słowo. Oznacza słowo które ma najwyższą wagę w liczbie.
Least Significant Word (LSW)	Najmniej znaczące słowo. Oznacza słowo które ma najniższą wagę w liczbie.
big endian	Konwencja zapisu danych, w której najbardziej znaczące słowo maszynowe (MSW) umieszczone jest jako pierwsze.
little endian	Konwencja zapisu, w której najmniej znaczące słowo maszynowe (LSW) znajduje się na początku.
Arytmetyka dowolnej precyzji (Bignum)	Metoda obliczeń komputerowych pozwalająca na operowanie na liczbach o rozmiarach ograniczonych jedynie dostępną pamięcią RAM, a nie rozmiarem rejestrów procesora.
Przepełnienie (<i>Overflow</i>)	Sytuacja, w której wynik operacji arytmetycznej wykracza poza zakres dostępny dla danego typu danych (np. poza 32 bity).
Bit przeniesienia (<i>Carry bit</i>)	Dodatkowy bit przechowujący informację o przepełnieniu podczas dodawania, przekazywany do operacji na następnym słowie maszynowym.
Interpolacja wielomianowa	Metoda wyznaczania wielomianu na podstawie zbioru punktów, wykorzystywana w algorytmach Toom-Cook do odzyskiwania współczynników iloczynu.
Benchmark	Test wydajnościowy pozwalający na porównanie szybkości działania różnych implementacji lub systemów w tych samych warunkach.

1 Wprowadzenie

Duże liczby, to znaczy liczby zawierające nawet ponad setki cyfr w notacji dziesiętnej, pełnią istotną rolę między innymi we współczesnej kryptografii oraz w obliczeniach naukowych. Przykładem może być algorytm kryptograficzny RSA, często wykorzystujący liczby o długości 2048 albo 4096 bitów, który może służyć do szyfrowania danych oraz tworzenia podpisów cyfrowych. Rola, jaką duże liczby pełnią w tych obliczeniach, sprawia, że kluczowe staje się użycie algorytmów, umożliwiających poprawne wykonywanie operacji arytmetycznych na tych liczbach w jak najkrótszym czasie.

Sposoby zapisu W naszej pracy skoncentrujemy się na dużych liczbach całkowitych. Istnieją różne sposoby zapisu takich liczb. Przykładowo, dodatnia liczba całkowita A może być przedstawiona jako

$$A = a_{n-1}\beta^{n-1} + a_{n-2}\beta^{n-2} + \dots + a_1\beta + a_0$$

gdzie β oznacza ustaloną bazę, a symbolami a_i oznaczono liczby całkowite z przedziału $[0, \beta)$. Takie zapisy liczb sprawia, że w pamięci komputera wystarczy przechowywać tylko długość n liczby oraz współczynniki a_i , ponieważ baza β zazwyczaj jest już wcześniej ustalona. Pracując na komputerze 32-bitowym, β przyjmuje wartość 2^{32} , co wynika z tego, że długość słowa maszynowego wynosi 32 bity. Analogicznie, na komputerze 64-bitowym podstawa β jest równa 2^{64} . Innymi popularnymi wyborami są 10^9 , 10^{19} albo 2^{53} .

Istnieje również kilka sposobów na przechowywanie informacji o znaku liczby całkowitej. W reprezentacji znak-moduł (po angielsku *sign-magnitude*) jeden wydzielony bit w zapisie liczby zawiera informację o jej znaku. Jeżeli jest on równy 0 to liczba jest nieujemna. Z kolei wartość 1 oznacza, że jest ona niedodatnia. Wartość bezwzględna liczby zawarta jest w pozostałych bitach użytych do przechowywania danej liczby. Wadą tego sposobu jest to, że 0 posiada w nim dwie reprezentacje. Jednym z rozwiązań tego problemu jest przyjęcie, że dana liczba jest równa 0 wtedy i tylko wtedy, gdy jej długość n jest równa 0, i w stosowanych algorytmach rozważać ten przypadek osobno.

W tej pracy omawiać będziemy implementację mnożenia liczb o dowolnej precyzji. Arytmetyka dowolnej precyzji, znana również jako arytmetyka bignum, jest sposobem wykonywania obliczeń komputerowych, których dokładność jest ograniczona jedynie dostępną pamięcią komputera. Umożliwia ona przeprowadzanie obliczeń na liczbach, których zapis wymaga więcej niż jednego słowa maszynowego. Operacje arytmetyczne wykonywane są na fragmentach zawierających pojedyncze słowa maszynowe. Implementacja algorytmów arytmetyki dowolnej precyzji wymaga znalezienia rozwiązań problemów takich jak powstawanie przepełnienia (*overflow*). Rozwiązaniem może być rozszerzenie liczby słów maszynowych, w jakich zapisywany jest wynik danej operacji arytmetycznej. Przykładem może być tu użycie bitów przeniesienia podczas dodawania dwóch liczb.

W przypadku kiedy zapis liczby wymaga więcej niż jednego słowa maszynowego, należy ustalić, w jakiej kolejności słowa te przechowywane będą w pamięci komputera. Dwa najważniejsze sposoby to tak zwane *big endian* oraz *little endian*. W pierwszej z tych konwencji najbardziej znaczące słowo maszynowe jest umieszczone jako pierwsze. W *little endian* odwrotnie, najmniej znaczące słowo umieszczone jest na pierwszej pozycji. Obecnie w obliczeniach arytmetycznych dominuje użycie konwencji *little endian*, ponieważ algorytmy wykonujące operacje arytmetyczne często rozpoczynają swoje działanie od najmniej znaczącego ze słów maszynowych, w których zapisana jest liczba.

Dodawanie i odejmowanie

Jedną z metod działania algorytmów mnożenia dużych liczb jest zmniejszenie liczby wykonywanych operacji mnożenia i zastąpienie części z nich operacjami dodawania, które cechują się mniejszym asymptotycznym tempem wzrostu.

Asymptotyczne tempo wzrostu to sposób na wyrażenie złożoności obliczeniowej danego algorytmu. Do wyrażenia tej wielkości często stosuje się notację dużego O . Została ona zdefiniowana poniżej.

Definicja 1. O funkcji f mówimy, że jest co najwyżej rzędu g gdy istnieją takie stałe $n_0 > 0$ oraz $c > 0$, że

$$\forall n \geq n_0 \quad f(n) \leq c \cdot g(n)$$

co zapisujemy $f(n) \in O(g(n))$.

Notacja ta może być interpretowana również jako koszt związany z liczbą wykonywanych instrukcji maszynowych albo, równoznacznie, koszt związany z czasem wykonywania algorytmu przez pojedynczy procesor, z dokładnością do stałej.

Podstawowym algorytmem wykonywania dodawania w arytmetyce o dowolnej precyzji jest tzw. dodawanie szkolne. Działa on analogicznie do dodawania pisemnego. Pseudokod tego algorytmu dla dwóch liczb o takiej samej długości został przedstawiony poniżej.

Algorithm 1: Dodawanie_szkolne

Input: $A = \sum_{i=0}^{n-1} a_i \beta^i$, $B = \sum_{i=0}^{n-1} b_i \beta^i$, bit przeniesienia $0 \leq d_{in} \leq 1$

Output: $C := \sum_{i=0}^{n-1} c_i \beta^i$, d takie, że $0 \leq d \leq 1$ oraz $A + B + d_{in} = C + d \cdot \beta^n$

```

1  $d \leftarrow d_{in}$ 
2 for  $i$  from 0 to  $n - 1$  do
3    $s \leftarrow a_i + b_i + d$ 
4    $(d, c_i) = (s \text{ div } \beta, s \text{ mod } \beta)$ 
5 return  $C, d$ 

```

Wartość sumy s może wynosić więcej niż wartość podstawy β . W celu jej zmniejszenia do poprawnie określonej wartości wykonywana jest operacja dzielenia z resztą przez β . Z kolei bitowi przeniesienia przypisywany jest wynik dzielenia całkowitego s przez β . To znaczy, w przypadku gdy s jest większe niż β , bit przeniesienia będzie równy 1.

Algorytm odejmowania działa podobnie jak ten służący do dodawania. Różnią się one tylko tym, że zamiast bitu przeniesienia wykorzystywane jest tak zwane pożyczanie $d \in \{-1, 0\}$. Drugą różnicą jest to, że zamiast sumy s liczona jest wartość wyrażenia $s \leftarrow a_i - b_i + d$. Cały pseudokod tego algorytmu został pokazany poniżej.

Algorithm 2: Odejmowanie

Input: $A = \sum_{i=0}^{n-1} a_i \beta^i$, $B = \sum_{i=0}^{n-1} b_i \beta^i$, pożyczanie $d \in \{-1, 0\}$

Output: $C := \sum_{i=0}^{n-1} c_i \beta^i$, d takie, że $d \in \{-1, 0\}$ oraz $A - B + d_{in} = C + d \cdot \beta^n$

```

1  $d \leftarrow d_{in}$ 
2 for  $i$  from 0 to  $n - 1$  do
3    $s \leftarrow a_i - b_i + d$ 
4    $(d, c_i) = (s \text{ div } \beta, s \text{ mod } \beta)$ 
5 return  $C, d$ 

```

Złożoność zarówno algorytmu dodawania, jak i odejmowania wynosi $O(n)$. Jest tak, ponieważ wraz z rosnącym rozmiarem n liczby liniowo rośnie liczba wykonywanych operacji obliczania wartości s . Złożoność ta jest zaniedbywalna w porównaniu ze złożonością podstawowego, szkolnego mnożenia, która wynosi $O(n^2)$. Dlatego właśnie w algorytmach szybkiego mnożenia mnożenie zastępowane jest dodawaniem oraz odejmowaniem.

Mnożenie

Podstawowym algorytmem służącym do mnożenia dwóch liczb jest tak zwane mnożenie szkolne albo mnożenie pisemne. Pseudokod tej metody został przedstawiony poniżej.

Algorithm 3: Mnożenie_szkolne

Input: $A = \sum_{i=0}^{n-1} a_i \beta^i$, $B = \sum_{j=0}^{m-1} b_j \beta^j$

Output: $C = A \cdot B := \sum_{i=0}^{n+m-1} c_i \beta^i$

```

1  $C \leftarrow A \cdot b_0$ 
2 for  $j$  from 0 to  $m - 1$  do
3    $C \leftarrow C + \beta^j \cdot (A \cdot b_j)$ 
4 return  $C$ 

```

W pierwszej i trzeciej linijce kodu wykonywane są mnożenia przez skalar. Złożoność pojedynczej takiej operacji zależy od rozmiaru liczby A i rośnie z nim liniowo. W celu wykonania pełnego mnożenia liczby A przez liczbę B liczba wykonywanych mnożeń przez skalar zależy od rozmiaru

liczby B i rośnie liniowo wraz z nim. Zatem złożoność całego algorytmu mnożenia pisemnego jest równa $O(n^2)$.

W kroku trzecim wyznaczana jest wartość podstawy β podniesionej do potęgi j -tej. Operacja ta może być wykonana poprzez zwyczajne przesunięcie o j słów maszynowych w stronę słowa najbardziej znaczącego.

Ponieważ mnożenie liczby przez skalar o długości jednego słowa maszynowego jest podstawową operacją wykonywaną przez powyższy algorytm, jednym ze sposobów na zwiększenie jego szybkości jest optymalizacja tego działania.

Pierwszy algorytm mnożenia dużych liczb o złożoności mniejszej niż $O(n^2)$ został odkryty w 1960 roku przez A. Karacubę. Wykorzystuje on zasadę dziel-i-rządź, to znaczy rozkłada większy problem na kilka mniejszych, łatwiejszych do rozwiązania. W przypadku algorytmu Karacuby mnożenie dwóch liczb o długości n słów wymaga trzech mnożeń liczb o długości $\frac{n}{2}$ oraz dodawania o zanedbywalnej złożoności. Poprzez powtarzanie takich podziałów osiągana jest asymptotyczna złożoność $n^{\log_2 3} \approx n^{1.585}$. Osiągana złożoność asymptotyczna jest więc niższa niż w przypadku mnożenia pisemnego.

Ponieważ algorytm ten jest bardziej skomplikowany niż podstawowy algorytm mnożenia, staje się on szybszy od zwykłego mnożenia pisemnego dopiero dla liczb o długości większej niż pewna długość progowa. Dokładna wartość tego progu zależy od używanego procesora oraz od kosztu względnego mnożenia i dodawania.

Poniżej przedstawiono pseudokod algorytmu Karacuby służący do mnożenia dwóch liczb o długości n . Symbolem n_0 oznaczono progową długość. Dla liczb o długości większej niż n_0 wykonywane jest mnożenie Karacuby, z kolei dla liczb o mniejszej długości wykonywane jest mnożenie szkolne.

Algorithm 4: Mnożenie_Karacuby

Input: $A = \sum_{i=0}^{n-1} a_i \beta^i$, $B = \sum_{j=0}^{n-1} b_j \beta^j$

Output: $C = A \cdot B := \sum_{i=0}^{2n-1} c_i \beta^i$

```

1 if  $n < n_0$  then
2   return Mnożenie_szkolne( $A, B$ )
3  $k \leftarrow \lceil \frac{n}{2} \rceil$ 
4  $(A_0, B_0) := (A, B) \bmod \beta^k$ ,  $(A_1, B_1) := (A, B) \operatorname{div} \beta^k$ 
5  $s_A \leftarrow \operatorname{sign}(A_0 - A_1)$ ,  $s_B \leftarrow \operatorname{sign}(B_0 - B_1)$ 
6  $C_0 \leftarrow \text{Mnożenie\_Karacuby}(A_0, B_0)$ 
7  $C_1 \leftarrow \text{Mnożenie\_Karacuby}(A_1, B_1)$ 
8  $C_2 \leftarrow \text{Mnożenie\_Karacuby}(|A_0 - A_1|, |B_0 - B_1|)$ 
9 return  $C := C_0 + (C_0 + C_1 - s_A \cdot s_B \cdot C_2) \cdot \beta^k + C_1 \cdot \beta^{2k}$ 
```

W powyższym algorytmie za podział jednej liczby na dwie o mniejszych długościach odpowiada trzecia i czwarta linia. Następnie, rekurencyjnie przeprowadzane są trzy operacje mnożenia na parach mniejszych liczb. W ostatnim kroku obliczany jest finalny iloczyn, z wykorzystaniem dodawania, odejmowania, mnożenia przez skalar oraz przesunięć bitowych. Operacje te mają niższą złożoność niż mnożenie dwóch dużych liczb, dlatego nie wpływają na dominujący składnik złożoności całego algorytmu.

Istnieją różne odmiany algorytmu Karacuby. Ta przedstawiona powyżej jest nazywana wersją z odejmowaniem, ponieważ obliczane są wartości wyrażeń $|A_0 - A_1|$ i $|B_0 - B_1|$. Innym przykładem może być tak zwana wersja z dodawaniem, w której zamiast tego obliczane są sumy $A_0 + A_1$ oraz $B_0 + B_1$. Jednakże wadą tej drugiej metody jest to, że podczas dodawania mogą powstawać przeniesienia. Jednym ze sposobów na radzenie sobie z tym problemem może być zarezerwowanie dodatkowego słowa maszynowego w pamięci komputera. Kolejnym może być wykonanie dodatkowych działań dodawania. Oba z tych rozwiązań wpłynęłyby jednak negatywnie na szybkość wykonywania mnożenia.

W pseudokodzie pokazanym powyżej widoczne jest pewne podobieństwo pomiędzy mnożeniem liczb całkowitych a mnożeniem wielomianów. Mianowicie, liczby na wejściu algorytmu Karacuby mogą być przedstawione w postaci wielomianów, tak jak pokazano poniżej

$$A = A_0 + A_1 x$$

$$B = B_0 + B_1 x$$

dla $x = \beta^k$. Liczba C może być wówczas obliczona jako iloczyn wielomianów A i B

$$C = (A_0 + A_1 x) \cdot (B_0 + B_1 x) = A_0 B_0 + (A_0 B_1 + A_1 B_0) x + A_1 B_1 x^2$$

wyznaczony w punkcie $x = \beta^k$. Iloczyn ten jest stopnia drugiego. Wiadomo również, że wielomian stopnia co najwyżej drugiego jest jednoznacznie wyznaczony przez wartości w trzech różnych punktach. W algorytmie mnożenia Karacuby podanym powyżej punkty te to $x = 0$ ($A = A_0, B = B_0$), $x = -1$ ($A = A_0 - A_1, B = B_0 - B_1$) oraz $x = \infty$ ($A = A_1, B = B_1$, obliczany jest iloczyn współczynników przy najwyższej potędze zmiennej x).

Generalizacją algorytmu Karacuby są algorytmy mnożenia Toom-Cook. Podczas gdy w mnożeniu Karacuby liczby dzielone są na dwa mniejsze fragmenty, w mnożeniu Toom-Cook- r następuje podział na r części o zbliżonej długości. Do opisanego algorytmu można również użyć przedstawionej powyżej analogii do interpolacji wielomianów. Mianowicie liczby A i B na wejściu mogą być utożsamione z wielomianami $A_0 + A_1x + \dots + A_{r-1}x^{r-1}$ oraz $B_0 + B_1x + \dots + B_{r-1}x^{r-1}$ dla $x = \beta^k$, gdzie $k = \lceil \frac{n}{r} \rceil$. Ich iloczyn $C(x)$ jest stopnia $2r - 2$, co oznacza, że w celu jego jednoznacznego wyznaczenia należy obliczyć jego wartość w $2r - 1$ punktach. Następnie możemy wyznaczyć iloczyn $C = A \cdot B$ poprzez podstawienie $x = \beta^k$ do wielomianu $C(x)$.

W algorytmie Toom-Cook- r jedno mnożenie liczb o długości n zamieniane jest na $2r - 1$ mnożeń liczb o długości około $\frac{n}{r}$. Złożoność takiego algorytmu zależy od tego, jakich punktów użyjemy do wyznaczenia wielomianu $C(x)$. W algorytmie Toom-Cook-3 punktami tymi są $0, 1, -1, 2$ oraz ∞ (analogicznie jak dla algorytmu Karacuby). Pseudokod tego algorytmu został pokazany poniżej. $n_1 \geq 3$ oznacza długość progową, dla liczb o długościach mniejszych niż n_1 stosowane jest mnożenie Karacuby.

Algorithm 5: Mnożenie_ToomCook3

Input: Dwie liczby całkowite $0 \leq A, B < \beta^n$

Output: $A \cdot B := c_0 + c_1\beta^k + c_2\beta^{2k} + c_3\beta^{3k} + c_4\beta^{4k}$ dla $k = \lceil \frac{n}{3} \rceil$

```

1 if  $n < n_1$  then
2   return Mnożenie_Karacuby( $A, B$ )
3 write  $A = a_0 + a_1x + a_2x^2, B = b_0 + b_1x + b_2x^2, x = \beta^k$ 
4  $v_0 \leftarrow \text{ToomCook3}(a_0, b_0)$ 
5  $a_{02} \leftarrow a_0 + a_2$ 
6  $b_{02} \leftarrow b_0 + b_2$ 
7  $v_1 \leftarrow \text{ToomCook3}(a_{02} + a_1, b_{02} + b_1)$ 
8  $v_{-1} \leftarrow \text{ToomCook3}(a_{02} - a_1, b_{02} - b_1)$ 
9  $v_2 \leftarrow \text{ToomCook3}(a_0 + 2a_1 + 4a_2, b_0 + 2b_1 + 4b_2)$ 
10  $v_\infty \leftarrow \text{ToomCook3}(a_2, b_2)$ 
11  $t_1 \leftarrow \frac{(3v_0 + 2v_{-1} + v_2)}{6} - 2v_\infty$ 
12  $t_2 \leftarrow \frac{v_1 + v_{-1}}{2}$ 
13  $c_0 \leftarrow v_0, c_1 \leftarrow v_1 - t_1, c_2 \leftarrow t_2 - v_0 - v_\infty, c_3 \leftarrow t_1 - t_2, c_4 \leftarrow v_\infty$ 
```

Za podział liczb na mniejsze części odpowiada trzecia linia powyższego pseudokodu. W kolejnych liniach wykonywana jest interpolacja wielomianu $C(x)$ w wymienionych wcześniej punktach. Wszystkie ilorazy obliczane w liniach jedenastej i dwunastej są liczbami całkowitymi. Dzielenie przez sześć w linii jedenastej można rozłożyć na dzielenie przez trzy i dzielenie przez dwa, które jest na tyle szybkie, że może być wykonane poprzez odpowiednie przesunięcie bitowe.

Dla liczb o jeszcze większych rozmiarach używa się algorytmów wykorzystujących szybką transformację Fouriera (fast Fourier transform, FFT). Przykładem może być algorytm Schönhage-Strassen o złożoności $O(n \log n \log \log n)$. Algorytm ten wykorzystuje arytmetykę modularną i działa w pierścieniu $\mathbb{Z}/(2^n + 1)\mathbb{Z}$.

Jak wspomniano wcześniej, algorytm Toom-Cook z założenia działa na dwóch liczbach o zbliżonej długości, dzieląc je na odpowiednią ilość części. W praktyce często zachodzi jednak potrzeba pomnożenia liczb o różnych rozmiarach (tzw. mnożenie niesymetryczne). Z powodu braku wsparcia dla asymetrii w standardowej wersji algorytmu wymusza on zastosowanie metod wyrównujących. W zaimplementowanej przez nas strukturze **WielkaLiczba** problem asymetrii rozwiązaliśmy poprzez metodę dodawania wiodących zer. Podejście to polega na wyrównaniu operacyjnej długości obu liczb przed przystąpieniem do podziału na wielomiany. W pierwszym kroku wyznaczamy wspólną długość n , która jest równa długości większej z mnożonych liczb:

$$n = \max(|A|, |B|)$$

Wartość n jest następnie używana do określenia rozmiaru pojedynczego fragmentu $k = \lceil \frac{n}{3} \rceil$. W przypadku krótszej liczby zastosowanie tego podziału powoduje, że najbardziej znaczące słowa byłyby puste.

Aby algorytm mógł wykonać poprawne obliczenia, braki te są uzupełniane wiodącymi zerami. Dzięki temu mniejsza liczba sztucznie osiąga rozmiar większej, co pozwala na zastosowanie standardowej procedury podziału na trzy części i wykonanie klasycznego algorytmu Toom-3. Metoda ta została przez nas zoptymalizowana w kodzie poprzez niejawnie traktowanie pustych przestrzeni jako zer w operacjach dodawania i dzielenia bloków (szczegóły w rozdziale 3), co pozwoliło uniknąć relokacji pamięci wektorów.

Choć nasze rozwiązanie wydaje się optymalne dla podstawowych potrzeb struktury **BigNum**, operowanie na pustych blokach wygenerowanych przez wiodące zera prowadzi do wykonywania nadmiarowych, pustych mnożeń.

W profesjonalnych bibliotekach, takich jak GMP (z którą porównujemy naszą wersję algorytmu), w przypadku dużej asymetrii stosuje się wydajniejsze techniki:

- **Warianty niesymetryczne algorytmu (np. Toom-Cook-(3,2)):** Zamiast wyrównywać liczby zerami, dłuższa liczba jest dzielona na więcej części niż krótsza. Pozwala to na zmniejszenie stopnia wielomianu wynikowego, a co za tym idzie – zredukowanie liczby potrzebnych punktów interpolacji i rekurencyjnych mnożeń.
- **Dekompozycja blokowa:** W przypadku różnicy długości liczb, dłuższa liczba jest dzielona na mniejsze części (bloki) o długości odpowiadającej krótszej liczbie. Cała operacja sprowadza się wówczas do wykonania serii niezależnych mnożeń tych mniejszych bloków przez krótszą liczbę, a następnie zsumowania wyników po ich odpowiednim przesunięciu bitowym.

2 Metodologia

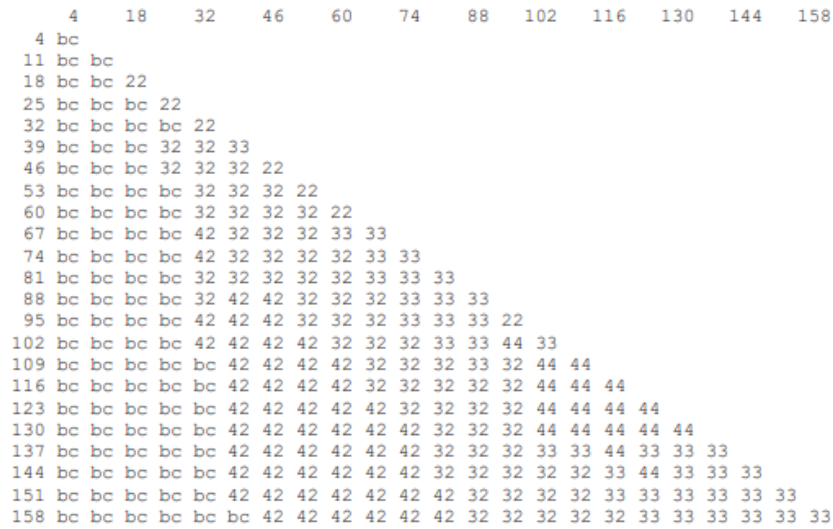
Cel eksperymentu

Celem eksperymentu jest zbadanie zależności pomiędzy czasem wykonania operacji mnożenia a rozmiarem danych wejściowych dla utworzonej implementacji algorytmu mnożenia dużych liczb. W drugiej części eksperymentu, na podstawie otrzymanej zależności, nastąpi porównanie naszej implementacji z istniejącymi bibliotekami.

Opis eksperymentu

Zbiór liczb użyty w eksperymencie będzie składał się z liczb w zapisie o podstawie $\beta = 2^{32}$. W eksperymencie zostaną użyte liczby z zakresu od 50-słowych do 1000-słowych (zakładamy, że słowo = 32 bity).

Będziemy mierzyć czas wykonania operacji mnożenia dla liczb o długości 50, 100, 150, ..., 950, 1000 słów. Dla każdej z podanych długości zostanie wygenerowanych 40 liczb o zadanej długości, następnie w parach (20 par dla każdej z ustalonych długości) zostanie przeprowadzone mnożenie. Zakres długości liczb został wybrany na podstawie wykresu 1, pokazanym poniżej.



Rysunek 1: The best algorithm to multiply two numbers of x and y words for $4 \leq x \leq y \leq 158$: *bc* is a schoolbook multiplication, *22* is Karatsuba's algorithm, *33* is Toom-3, *32* is Toom-(3,2), *44* is Toom-4, and *42* is Toom-(4,2). This graph was obtained on a Core 2, with GMP 5.0.0, and GCC 4.4.2. Note that for $x \leq (y+3)/4$, only the schoolbook multiplication is available; since we did not consider the algorithm that cuts the larger operand into several pieces, this explains why *bc* is best for say $x = 32$ and $y = 158$. *Modern Computer Arithmetic*

Następnie zostanie zbadana zależność czasu potrzebnego do przeprowadzenia operacji mnożenia w zależności od tego, jak duże są mnożone liczby dla utworzonej przez nas implementacji.

W kolejnym kroku te same pary liczb zostaną pomnożone z wykorzystaniem biblioteki GMP (GNU Multiple Precision Arithmetic Library) oraz w języku Python. Umożliwi to wyznaczenie analogicznej zależności jak dla naszej implementacji oraz ich porównanie.

Plan osiągnięcia celów projektowych

Szkielet implementacji

- Przygotowanie specyfikacji – opis struktury programu, reprezentacji liczb BigInt oraz wymagań dla algorytmu Toom-3, związanych z metodologią.
- Generowanie wstępnego szkieletu kodu – wykorzystanie modelu Gemini do wygenerowania podstawowej struktury projektu w C++.
- Zapisywanie wszystkich użytych promptów oraz otrzymanych odpowiedzi (prompt documentation).
- Ręczna weryfikacja oraz przygotowanie do dalszej implementacji algorytmów.

Eksperyment i benchmarki

- Przygotowanie próbek – z użyciem gotowej biblioteki do generowania liczb pseudolosowych i zapisu danych wejściowych według ustalonej metodologii.
- Testy własnej implementacji – pomiar czasu mnożenia próbek.
- Testy rozwiązań GMP - benchmark w porównaniu do profesjonalnego standardu C++.
- Testy rozwiązań wbudowanych w Pythonie - pomiar czasu w Pythonie.
- Zapisanie wyników - względem ustalonej metodologii (.csv).
- Połączenie wszystkich programów, wykorzystując narzędzie CMake.

Analiza wyników

- Wykresy i analiza danych – wykorzystanie bibliotek takich jak scipy, numpy, matplotlib, seaborn, pandas.
- Wnioski końcowe – podsumowanie efektywności algorytmu w praktyce.

3 BigInt

Użycie Sztucznej Inteligencji Jak wspomniano wcześniej, nasza implementacja BigInt oraz algorytm mnożenia Toom-Cooka 3 (Toom-3) będą wykonane za pomocą narzędzi AI. Zdecydowano się na wykorzystanie modelu językowego Google Gemini 1.5 Pro, którego przeznaczeniem jest praca programistyczna oraz zagadnienia matematyczne.

Rola modelu obejmowała wygenerowanie szkieletu struktury w języku C++ dla liczb całkowitych o dowolnej precyzji oraz implementację samych algorytmów. Część decyzji - między innymi to, w jaki sposób przetrzymywane są obiekty BigInt w pamięci, została podjęta przez nas. Cała dokumentacja interakcji z modelem językowym została zawarta w rozdziale 7.1.

Decyzje o implementacji Zaprojektowana struktura bazuje na kontenerze `std::vector` z biblioteki standardowej C++, przechowującym elementy typu `uint32_t` (32-bitowe liczby całkowite bez znaku). Zaimplementowano dwie wersje. Jedna przechowuje słowa w kolejności big endian, druga natomiast w kolejności little endian. Więcej szczegółów poniżej. W dalszej pracy oraz w samym eksperymencie korzystano z drugiej wersji, ponieważ jest ona bardziej wydajna czasowo.

Wygenerowany kod Model AI poprawnie wygenerował szkielet BigInt według naszych zaleceń. Został przez nas przejrany i zweryfikowany, a kompilator nie wyrzucał błędów. Zostały przez nas wykonane testy samych algorytmów, sprawdzające, czy wyniki są poprawne. Wszystkie rezultaty były pozytywne.

Do kodu dopisano stworzone przez nas funkcje, które mierzą czas wykonywania operacji mnożenia i zapisują dane do pliku tekstowego w ustalonym przez nas wcześniej formacie, w celu przeprowadzenia głównego eksperymentu.

Dwie wersje implementacji

Algorytm Toom-3 w dużej części polega na wstawianiu *zer wiodących*, czyli wstawianiu zer przed cyframi najbardziej znaczącymi.

$$456_{10} \xrightarrow{+\text{wiodące zero}} 0456_{10}$$

Gdy przechowujemy cyfry w formacie *big endian* w wektorze, czyli najbardziej znacząca cyfra jest na pierwszej pozycji tablicy, to wiodące zero jest wstawiane na początek wektora, co ma złożoność $O(n)$ (liniową). Złożoność liniowa wewnątrz pętli algorytmów mnożenia doprowadza do złożoności samego dodawania zer do $O(n^2)$.

Gdy jednak weźmiemy format *little endian*, czyli najbardziej znacząca cyfra jest ostatnim elementem tablicy, to wiodące zero jest wstawiane na koniec wektora. Wstawianie na koniec wektora ma złożoność $O(1)$ (stałą), co jest znacznie szybsze w porównaniu do złożoności liniowej.

Pierwsza wersja Zostały zrobione dwie implementacje, ponieważ pierwsza wersja korzystała z formatu big endian, co było naszym nieświadomym **błędem** i niedopatrzaniem na etapie samego planowania BigInt oraz zrozumienia algorytmu Toom-3.

Implementacja	Czas wykonania
Biblioteka Python (wbudowana)	$\approx 470 \mu s$
Pierwsza implementacja C++	$\approx 48 ms$

Tabela 1: Porównanie czasu wykonania mnożenia (liczby o długości 1000 słów 32-bitowych)

Błąd został zauważony przy pierwszych testach. W tabeli 1 widać, że nasza implementacja była w tym przypadku 100-krotnie wolniejsza od interpretera języka Python, co jest wynikiem znacząco odbiegającym od naszych oczekiwań.

Po zrozumieniu błędu została stworzona całkowicie nowa struktura. Druga implementacja przechowuje liczby w formacie little endian. Program dalej wczytuje z pliku liczby w formacie big endian (tak jak zdecydowano w metodologii), ale ich kolejność jest odwracana przy tworzeniu obiektu, co nie ma wpływu na sam czas działania algorytmu mnożenia.

Zmiana struktury przechowywania danych pozwoliła na znaczną poprawę wydajności.

Pozostała różnica względem rozwiązań profesjonalnych (np. GMP) wynika z braku niskopoziomowych optymalizacji, takich jak:

Implementacja	Czas wykonania
Biblioteka Python (wbudowana)	$\approx 470 \mu s$
Druga implementacja C++	$\approx 8 \text{ ms}$

Tabela 2: Porównanie czasu wykonania mnożenia (1000 słów 32-bitowych)

- **Niewykorzystanie pełnej szerokości rejestrów:** Zastosowanie słów 32-bitowych na architekturze 64-bitowej nie wykorzystuje w pełni mocy obliczeniowej procesora. Biblioteki klasy GMP operują na 64-bitowych słowach [3], co dwukrotnie redukuje liczbę wymaganych operacji elementarnych.
- **Zarządzanie pamięcią:** `std::vector` dokonuje częstych reallokacji. Profesjonalne biblioteki stosują *pre-allocation*, rezerwując pamięć na wynik operacji z góry.

Testy złożoności

Został stworzony kod w C++ w pliku o nazwie **fast_test.cpp**, który obrazuje, jak dane rozwiązania wpływają na czas. Można samodzielnie sprawdzić, jak czasowo różnią się wykonane operacje.

Reprezentacja	Liczba operacji	Czas wykonania
big endian (insert na początku)	100 000	$\approx 477,1 \text{ ms}$
little endian (push_back)	100 000	$\approx 3,8 \text{ ms}$

Tabela 3: Porównanie wydajności dopisywania elementów w zależności od reprezentacji

Konstruktor Dodatkowo, pierwsza implementacja tworzyła obiekty typu **WielkaLiczba** poprzez podawanie ich przez wartość zamiast przez referencję. Algorytm Toom-3 opiera się na tworzeniu wielu obiektów tego typu. Zostało to naprawione i można również zobaczyć różnice za pomocą pliku `fast_test.cpp`.

```
// BŁĄD: Niepotrzebne kopiowanie całego wektora
WielkaLiczba(std::vector<uint32_t> liczba) {
    słowa = liczba;
}
```

Wersja konstruktora	Czas wykonania
Bez referencji i const	$\approx 29 \text{ ms}$
Z referencją i const	$\approx 16 \text{ ms}$

Tabela 4: Czas utworzenia 100 000 obiektów struktury **WielkaLiczba**

4 GMP

Jednym z dostępnych gotowych rozwiązań dla mnożenia dużych liczb jest biblioteka GMP (*GNU Multiple Precision Arithmetic Library*). Biblioteka ta została wybrana, ponieważ ma status standardu na rynku w zakresie obliczeń dowolnej precyzji. Jest ona wykorzystywana w systemach algebry komputerowej (np. Mathematica, Maple) oraz w bibliotekach kryptograficznych [1]. Jest wolnym oprogramowaniem i można z niej korzystać według własnych potrzeb (opiera się na licencjach GNU).

GMP charakteryzuje się między innymi:

- **Wysoką optymalizacją niskopoziomową:** Kluczowe pętle obliczeniowe są napisane w assemblerze, specyficznym dla danej architektury procesora CPU.
- **Zaawansowanymi algorytmami:** Biblioteka automatycznie dobiera algorytm mnożenia (np. Karatsuba, Toom-Cook, FFT) w zależności od rozmiaru danych wejściowych oraz samego procesora, co czyni ją niezwykle trudną do prześcignięcia.

GMP pozwala zatem w obiektywny sposób ocenić inne implementacje jako rozwiązanie o możliwie maksymalnej wydajności.

Sposób korzystania z GMP Program został napisany na podstawie dokumentacji technicznej [3]. Skrypt wczytuje liczby z plików tekstowych do tablic dynamicznych. Są one następnie konwertowane do typu `mpz_t`, czyli dużych liczb całkowitych biblioteki GMP.

Program wykonuje mnożenia za pomocą wyłącznie funkcji dostarczonych przez GMP. Użyty algorytm mnożenia może różnić się w zależności od specyfikacji danego komputera. Najprawdopodobniej w naszym przypadku stosowany jest algorytm Toom-3 (bądź inne jego wersje), co sugeruje nam wybrany w metodologii przedział rozmiarów liczb.

Zadbano, by pamięć po przechowywanych liczbach była bezpiecznie zwalniana, co jest kluczowe dla wydajności programu. Biblioteka GMP nie zwalnia pamięci sama, więc należy o to zadbać. To jest kluczowe, gdy pracujemy z dużą ilością liczb.

5 Implementacja w języku Python

Python posiada wbudowaną obsługę liczb całkowitych dowolnej precyzji. Jest ona silnie zoptymalizowana i napisana bezpośrednio w języku C.

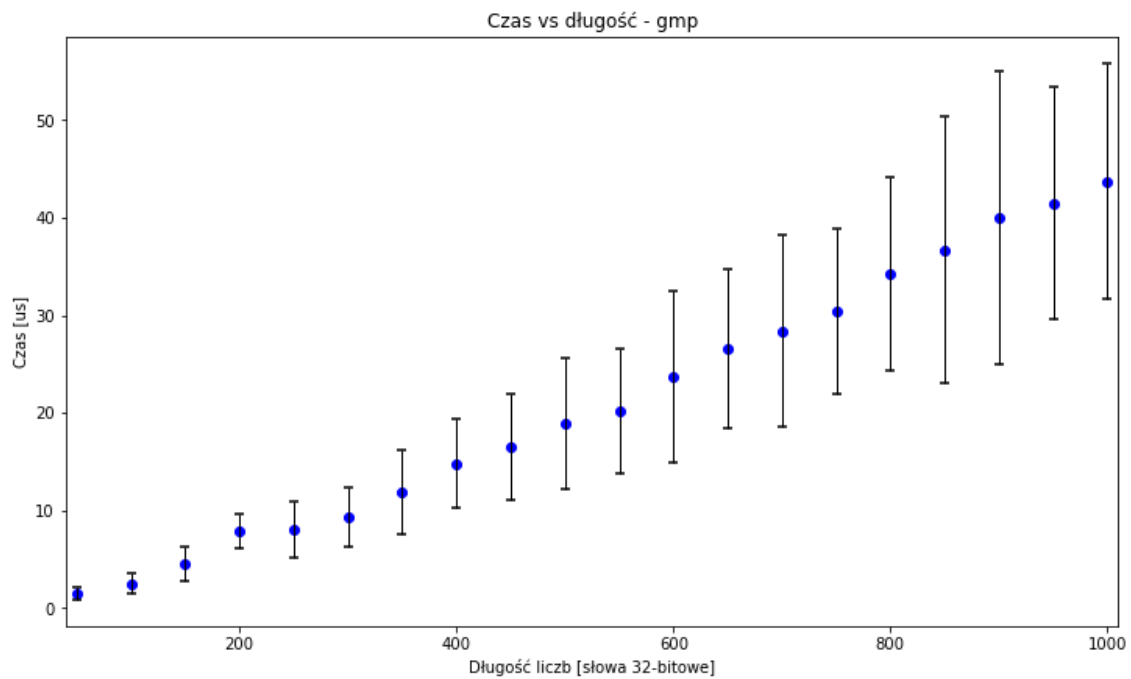
Zdecydowano się na wykorzystanie Pythona jako punkt odniesienia z racji prostoty obsługi dużych liczb oraz samej popularności tego języka. Dodatkowo pozwala on ocenić wydajność naszej implementacji.

Dane wejściowe w formacie big endian są wczytywane przez skrypt i konwertowane na obiekt `int` Pythona. Czas mierzy tylko samo mnożenie liczb. Następnie wyniki są zapisywane w takim samym formacie jak w poprzednich rozwiązaniach.

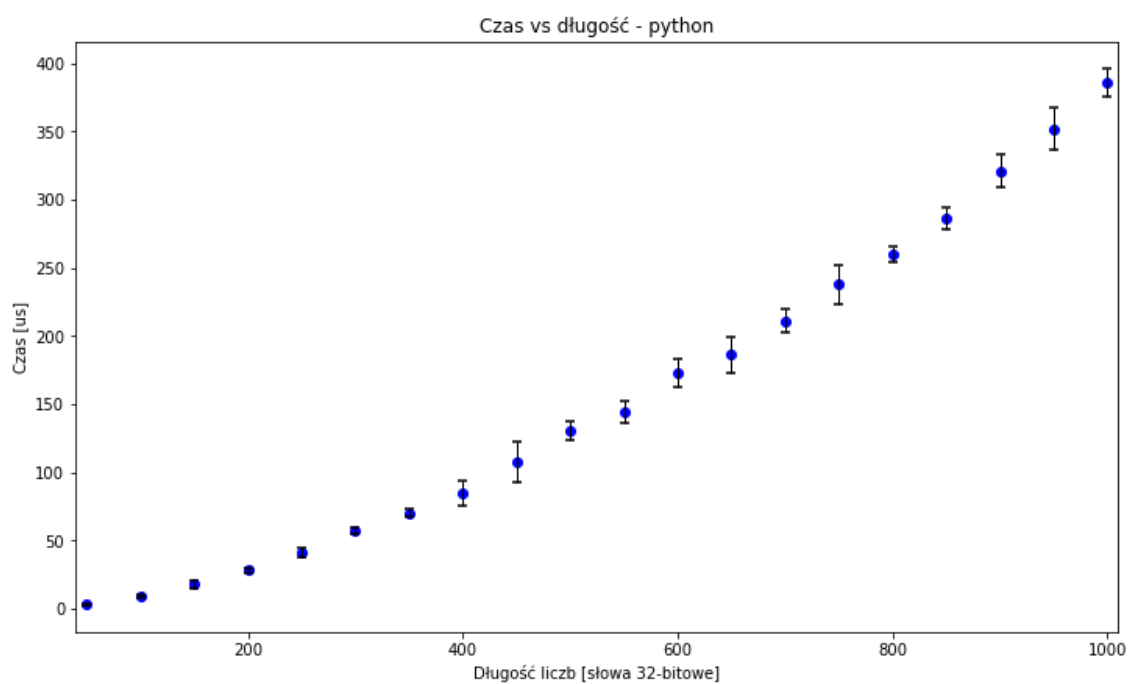
6 Analiza uzyskanych wyników

Zależność czasu od długości liczby

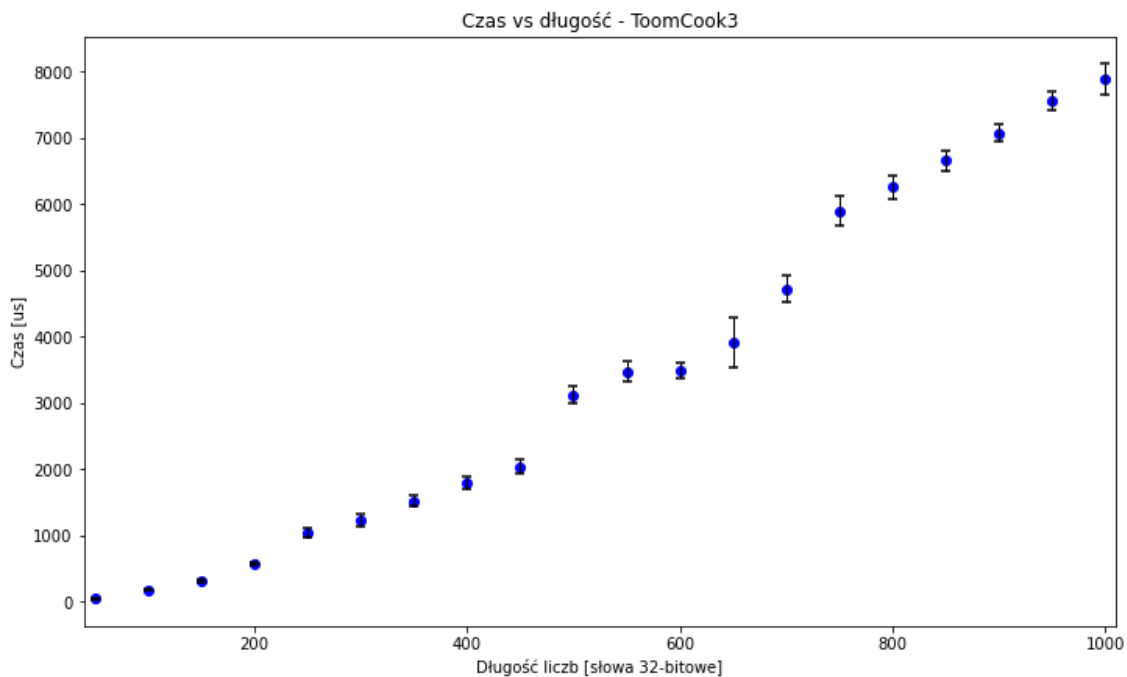
Poniżej przedstawiono wykresy przedstawiające uzyskane zależności średniego czasu potrzebnego do wykonania operacji mnożenia w zależności od rozmiaru liczby dla biblioteki GMP, rozwiązań wbudowanych w języku programowania Python oraz dla utworzonej implementacji algorytmu Toom-Cook-3.



Rysunek 2: Uzyskana zależność czasu potrzebnego do mnożenia w zależności od rozmiaru liczby dla biblioteki GMP.



Rysunek 3: Uzyskana zależność czasu potrzebnego do mnożenia w zależności od rozmiaru liczby dla języka Python.



Rysunek 4: Uzyskana zależność czasu potrzebnego do mnożenia w zależności od rozmiaru liczby dla utworzonej implementacji Toom-3.

Na każdym z powyższych wykresów niebieskie kropki oznaczają wyznaczone mediany czasów. Z kolei czarnymi słupkami oznaczono odchylenia standardowe dla zebranych danych.

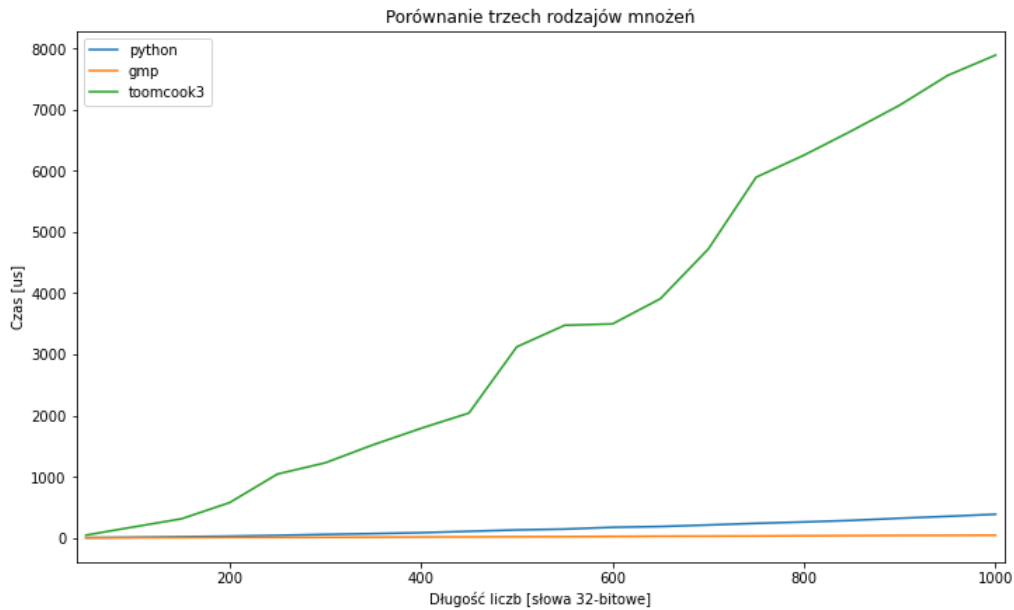
We wszystkich przypadkach widoczne są wyraźne rosnące zależności. Oznacza to, zgodnie z naszymi przypuszczeniami, że wraz z rosnącą długością liczb potrzebne jest więcej czasu, aby wykonać na nich operację mnożenia. W przypadku wykresu przedstawiającego wyniki dla utworzonej implementacji widoczny jest gwałtowny skok dla długości liczb równej około 500 słów 32-bitowych. Może on być spowodowany nieoptymalnie dobranym progiem przełączania się algorytmów albo przekroczeniem pamięci cache procesora. Kolejnym ciekawym elementem jest to, że dla niektórych długości liczb, odchylenia standardowe są wyraźnie większe w porównaniu z innymi. Może to wynikać z faktu, że dana długość nie jest podzielna przez trzy, co powoduje, że podczas obliczeń liczba dzielona jest na części o różnej długości. To z kolei może wpływać na zmianę w liczbie wykonywanych mnożeń i dodawań pomocniczych. Jednakże nie mamy co do tego pewności. W celu zweryfikowania tych przypuszczeń należy przeprowadzić kolejne eksperymenty.

Porównanie różnych metod wykonywania mnożenia

Na kolejnym wykresie przedstawiono porównanie, ile czasu zajmowało wykonywanie operacji mnożenia dla każdej z badanych długości liczb, w zależności od sposobu, w jaki to mnożenie zostało przeprowadzone.

Zgodnie z naszymi oczekiwaniami, mnożenie liczb przy użyciu utworzonej implementacji ToomCook-3 okazało się najwolniejsze. Najszybszą metodą, zgodnie z naszymi przypuszczeniami, okazała się biblioteka GMP. Jest tak, dlatego że została ona zaprojektowana do wysokowydajnej arytmetyki dużych liczb. Wykorzystuje ona między innymi niskopoziomowe optymalizacje pracy procesora oraz efektywniejsze zarządzanie pamięcią. Ponadto progi przełączania różnych algorytmów mnożenia są dynamicznie dostosowywane do aktualnych zadań. Są to również powody, dla których GMP jest szybsze niż rozwiązania wbudowane w języku Python.

O wpływie zarządzania pamięcią na szybkość wykonywania operacji mnożenia przekonaliśmy się również podczas pracy nad tworzoną implementacją (patrz rozdział 3). Testowana wersja algorytmu, w której zapis liczb zaczynał się od najbardziej znaczącego słowa, wykonywała mnożenie w czasie o rząd wielkości większym niż ostateczna wersja, która korzysta z liczb przechowywanych w pamięci komputera w odwrotnej kolejności. Zmiana sposobu zapisu liczb umożliwiła zmniejszenie czasu, który przeznaczony był na dostęp do najmniej znaczących słów maszynowych składających się na zapis liczby.



Rysunek 5: Porównanie metod mnożenia.

Dopasowanie modelu do danych

Kolejnym krokiem naszej analizy było sprawdzenie, czy uzyskane dane zgadzają się z teoretycznym modelem. Sprawdzenie tego dokonano na cztery sposoby, które zostały przedstawione poniżej.

1. Dopasowanie modelu o równaniu $y = a \cdot x^n$ do nie przekształconych danych
2. Dopasowanie modelu o równaniu $\log y = n \cdot \log x + \log a$ do przekształconych danych.
3. Dopasowanie modelu o teoretycznym asymptotycznym tempie wzrostu $n = 1,465$ do nie przekształconych danych
4. Dopasowanie modelu o teoretycznym asymptotycznym tempie wzrostu $n = 1,465$ do przekształconych danych.

Omawianą transformacją lub przekształceniem danych było wzięcie logarytmu zarówno ze średnich czasów potrzebnych na przeprowadzenie operacji mnożenia, jak i z długości liczb wyrażonych w liczbie słów 32-bitowych. Przekształcenie to pozwala na niestabilne numerycznie obliczenia podczas dopasowywania modelu do danych. Poprzez transformację logarytmiczną unika się działania potęgowania, które w omawianej sytuacji może prowadzić do powstawania dużych niedokładności i błędów.

Parametrami, które pozwoliły nam oszacować jakość dopasowania modelu do danych, były

- Współczynnik determinacji R^2 dany jest wzorem:

$$R^2 = \frac{\sum_{i=1}^n (\hat{y}_i - y_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

y_i oznacza wartość rzeczywistą, $\bar{y}$ to wartość średnia, $\hat{y}_i$ to wartość przewidywana przez model. Współczynnik ten mówi o tym, jaka część zmienności zmiennej zależnej jest wyjaśniana przez model.

- Błąd średniokwadratowy (MSE) dany jest wzorem

$$MSE = \mathbb{E}((y_i - \hat{y})^2)$$

Jest on równy wartości oczekiwanej kwadratów różnic pomiędzy wartością prawdziwą a przewidywaną przez model

- Pierwiastek z błędu średniokwadratowego jest dany wzorem

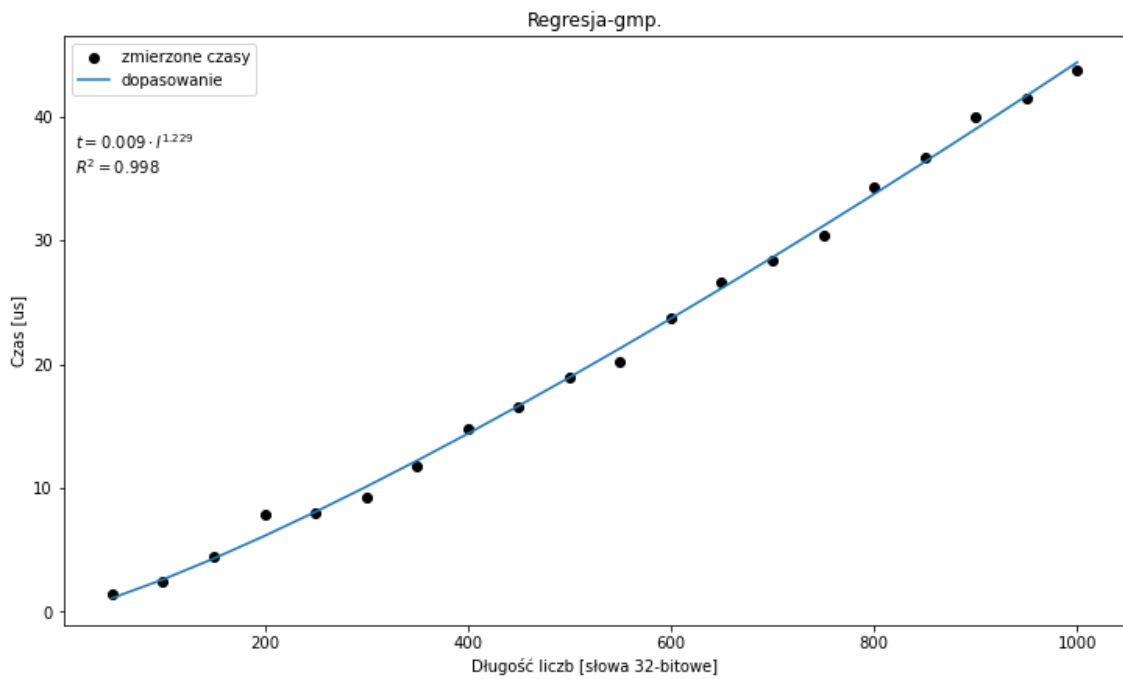
$$RMSE = \sqrt{MSE}$$

Parametr ten mierzy przeciętny rozmiar błędu w tej samej jednostce co dane.

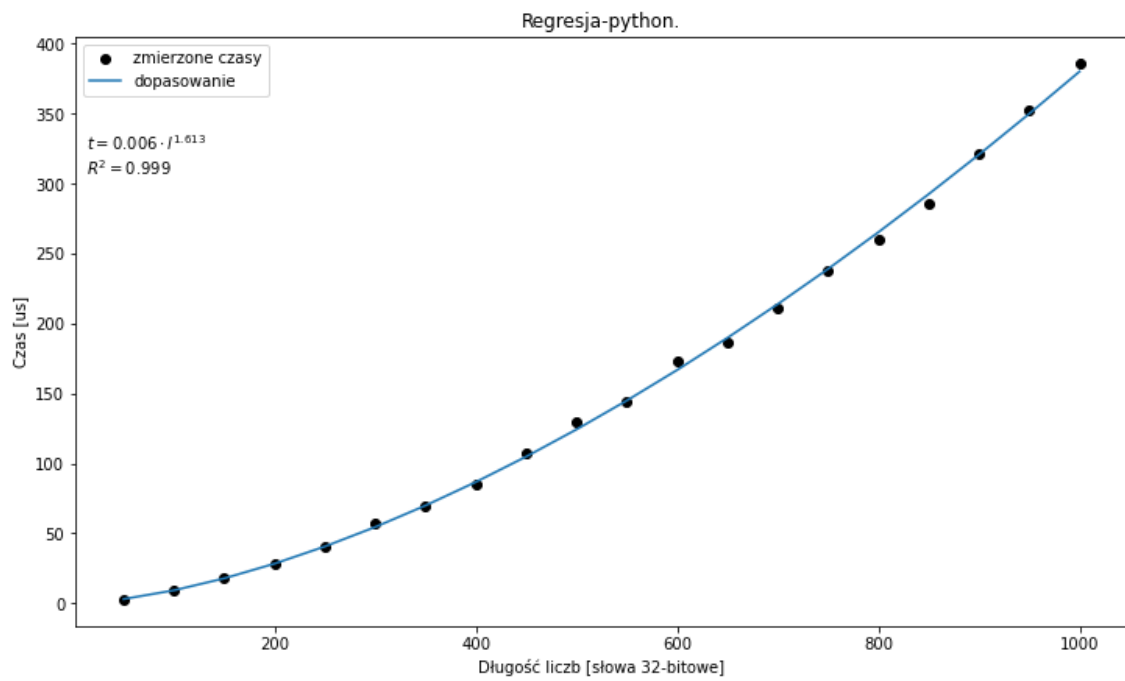
Wszystkie uzyskane parametry zostały przedstawione w poniższej tabeli. Nazwy poszczególnych sposobów odnoszą się do oznaczeń na przedstawionych poniżej wykresach. Analiza otrzymanych wyników nastąpi po przedstawieniu wszystkich uzyskanych wykresów.

Nazwa	R^2	MSE	$RMSE$
a)	0,9992	$1,1 \cdot 10^1$	3,3
b)	0,9977	$4 \cdot 10^{-1}$	0,6
c)	0,9908	$6,2 \cdot 10^4$	249,8
d)	0,9992	$3,7 \cdot 10^4$	191,1
e)	0,9958	$4,7 \cdot 10^4$	21,7
f)	0,9836	$2,0 \cdot 10^7$	4445,5
g)	0,9836	2,9	1,7
h)	0,9956	$6,0 \cdot 10^1$	7,8
i)	0,9893	$7,3 \cdot 10^4$	269,6
j)	0,9300	$6,0 \cdot 10^{-2}$	0,2
k)	0,9910	$1,5 \cdot 10^{-2}$	0,1
l)	0,9742	$4,8 \cdot 10^{-2}$	0,2

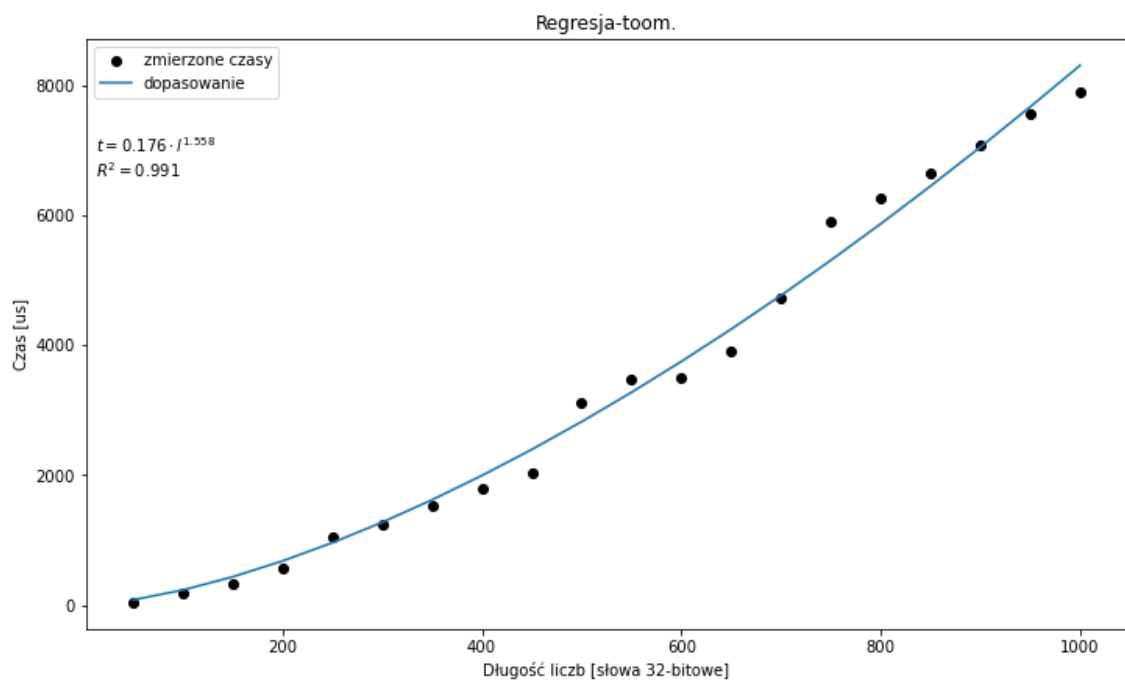
Tabela 5: Wartości parametrów opisujących jakość dopasowania modeli.



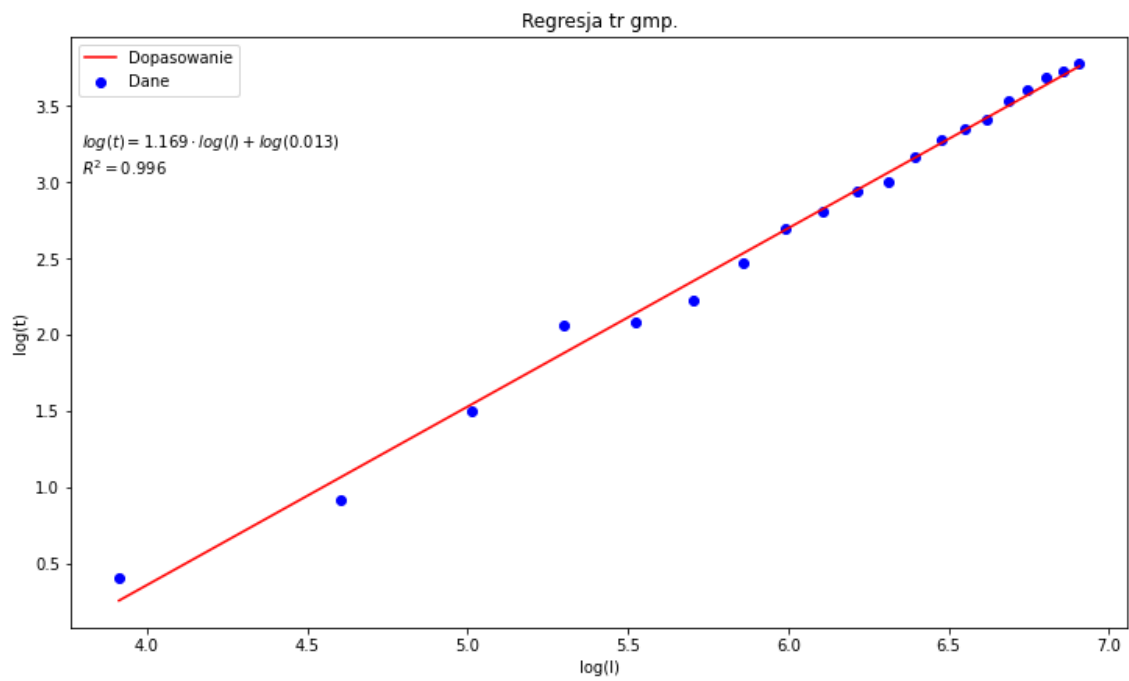
Rysunek 6: Dopasowanie modelu $y = ax^n$ do nieprzekształconych danych dla biblioteki GMP.



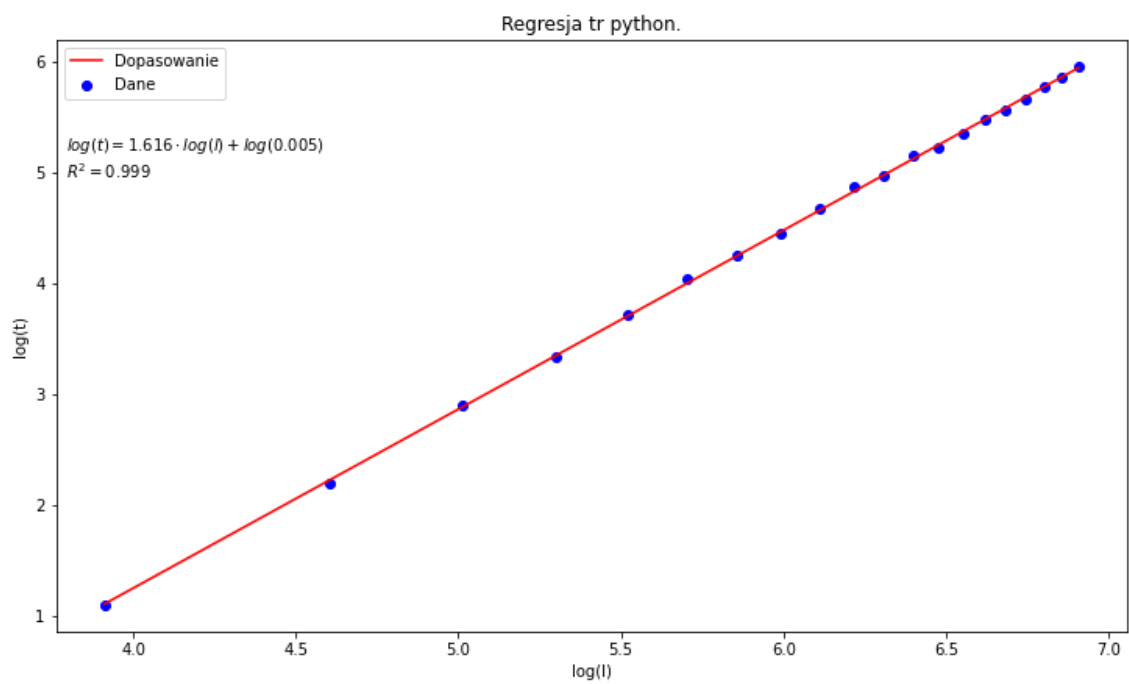
Rysunek 7: Dopasowanie modelu $y = ax^n$ do nieprzekształconych danych dla języka Python.



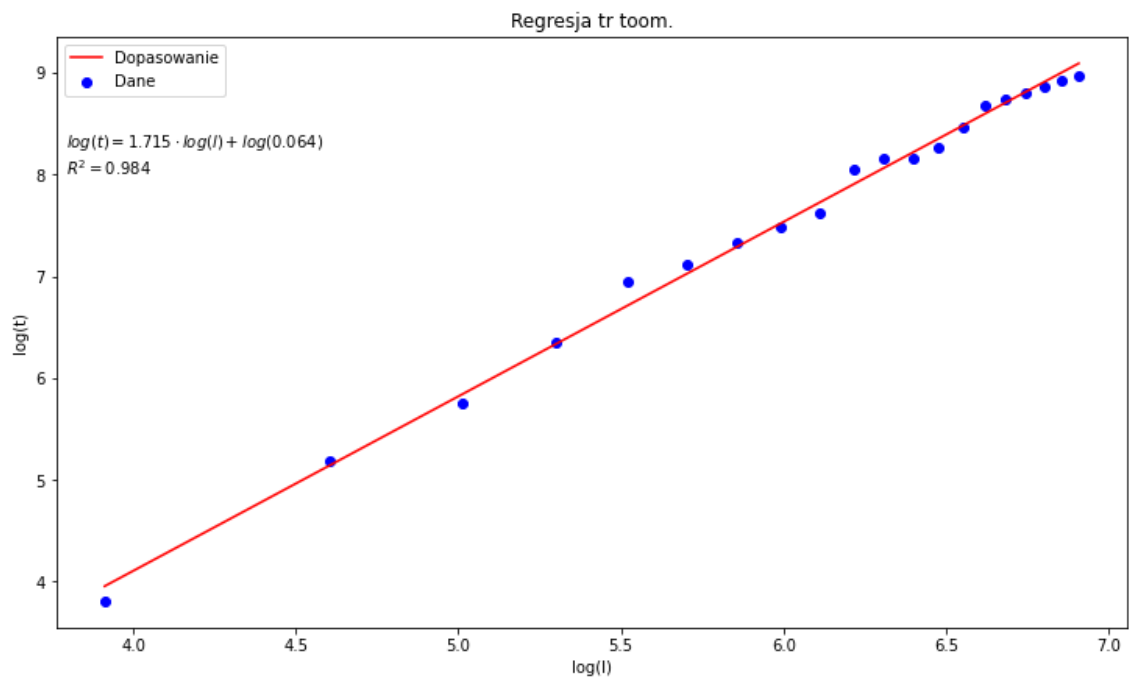
Rysunek 8: Dopasowanie modelu $y = ax^n$ do nieprzekształconych danych dla utworzonej implementacji.



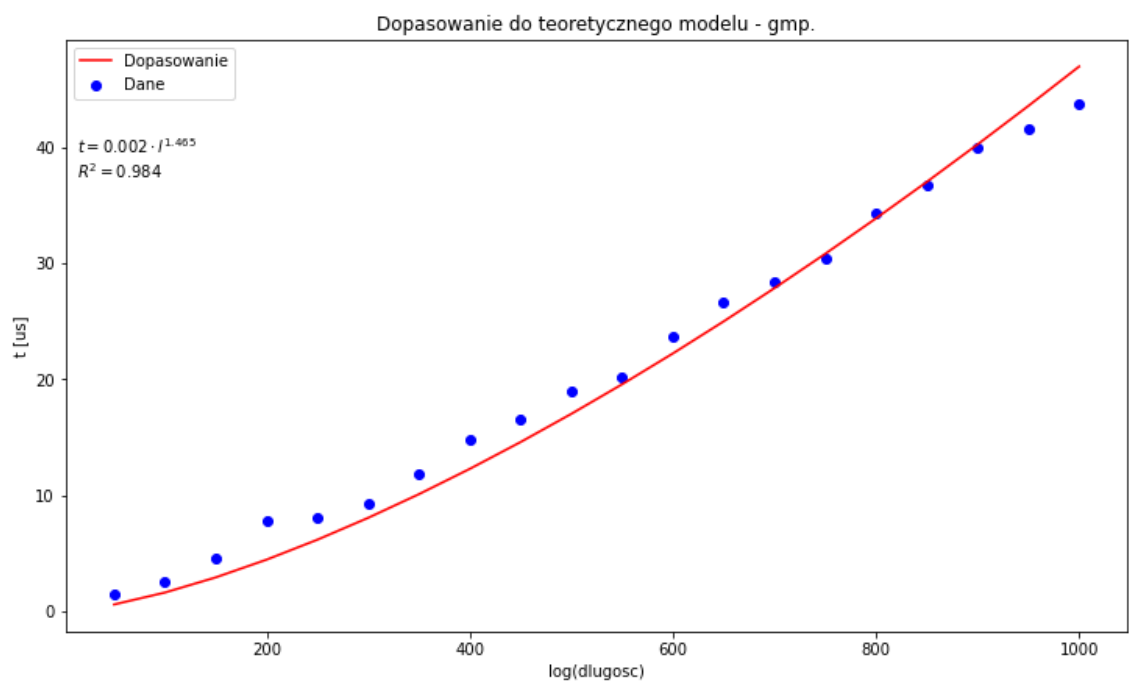
Rysunek 9: Dopasowanie modelu o równaniu $\log y = n \cdot \log x + \log a$ do przekształconych danych dla biblioteki GMP.



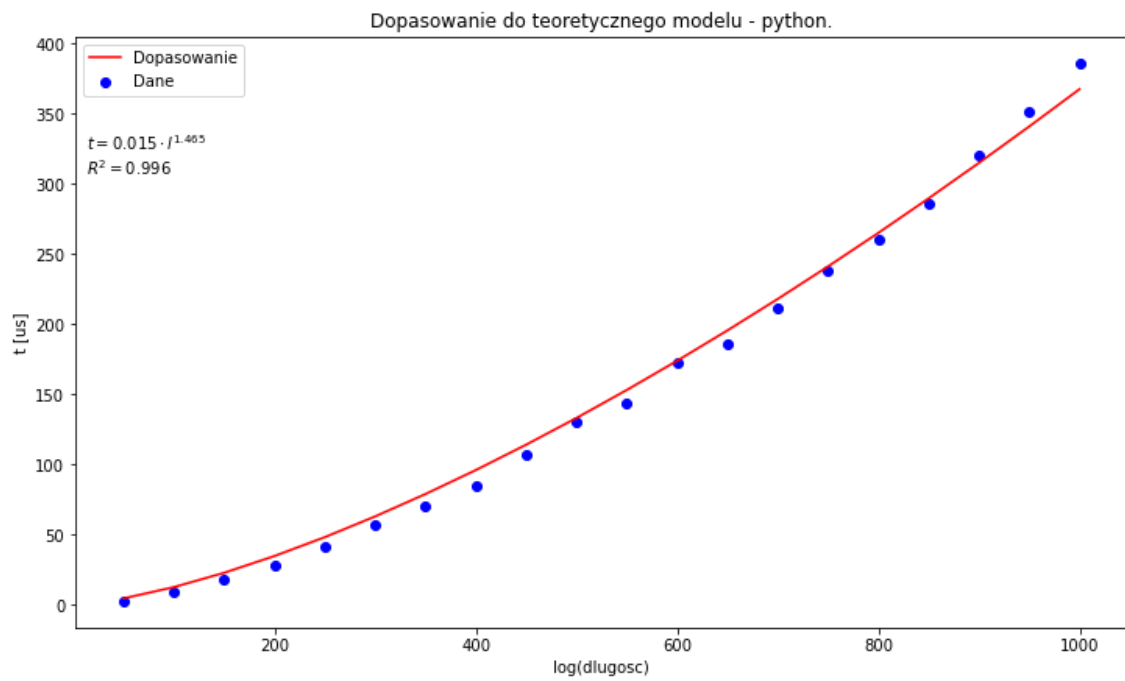
Rysunek 10: Dopasowanie modelu o równaniu $\log y = n \cdot \log x + \log a$ do przekształconych danych dla języka Python.



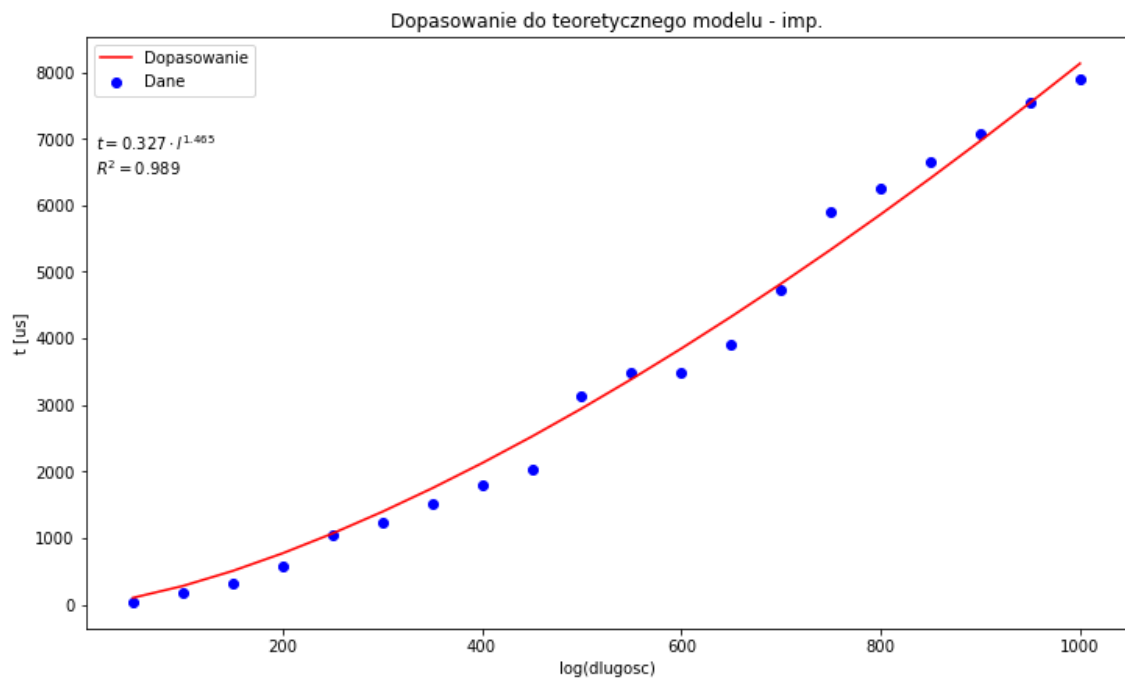
Rysunek 11: Dopasowanie modelu o równaniu $\log y = n \cdot \log x + \log a$ do przekształconych danych dla utworzonej implementacji.



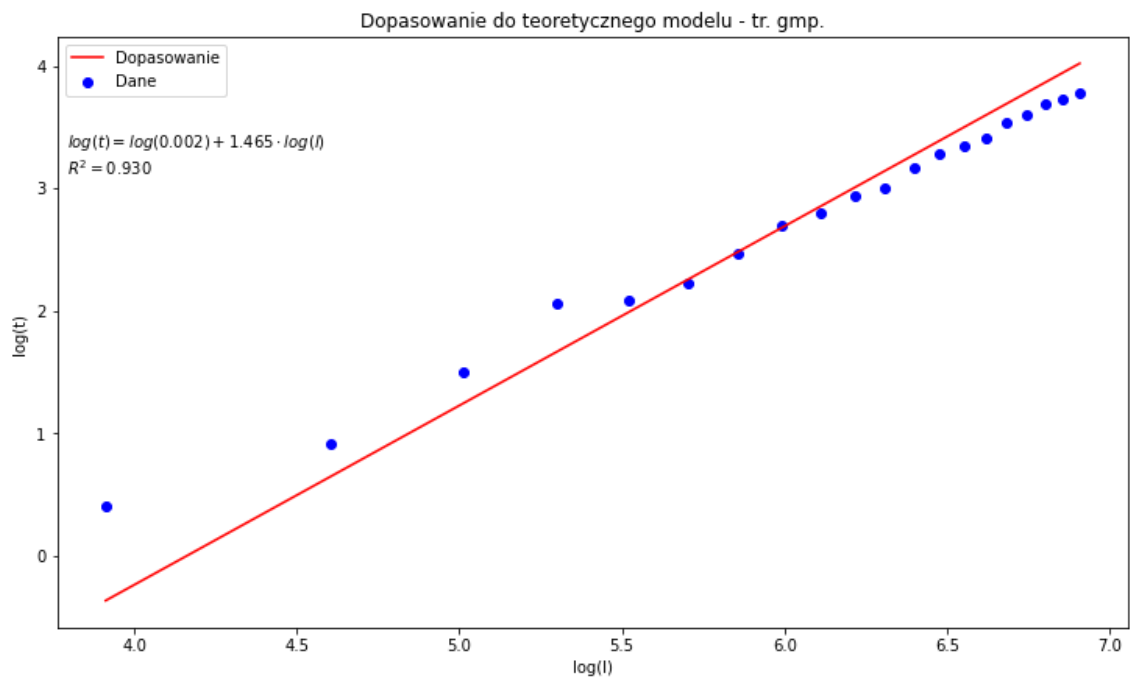
Rysunek 12: Dopasowanie modelu o teoretycznym asymptotycznym tempie wzrostu $n = 1,465$ do nieprzekształconych danych dla biblioteki GMP.



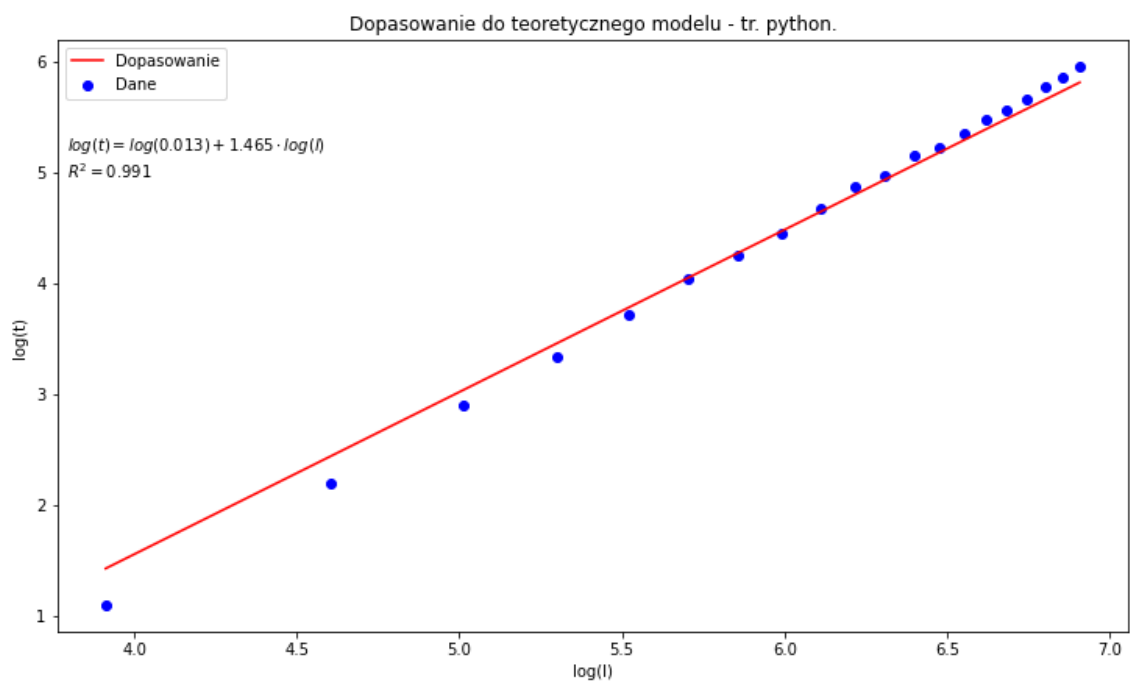
Rysunek 13: Dopasowanie modelu o teoretycznym asymptotycznym tempie wzrostu $n = 1,465$ do nieprzekształconych danych dla języka Python.



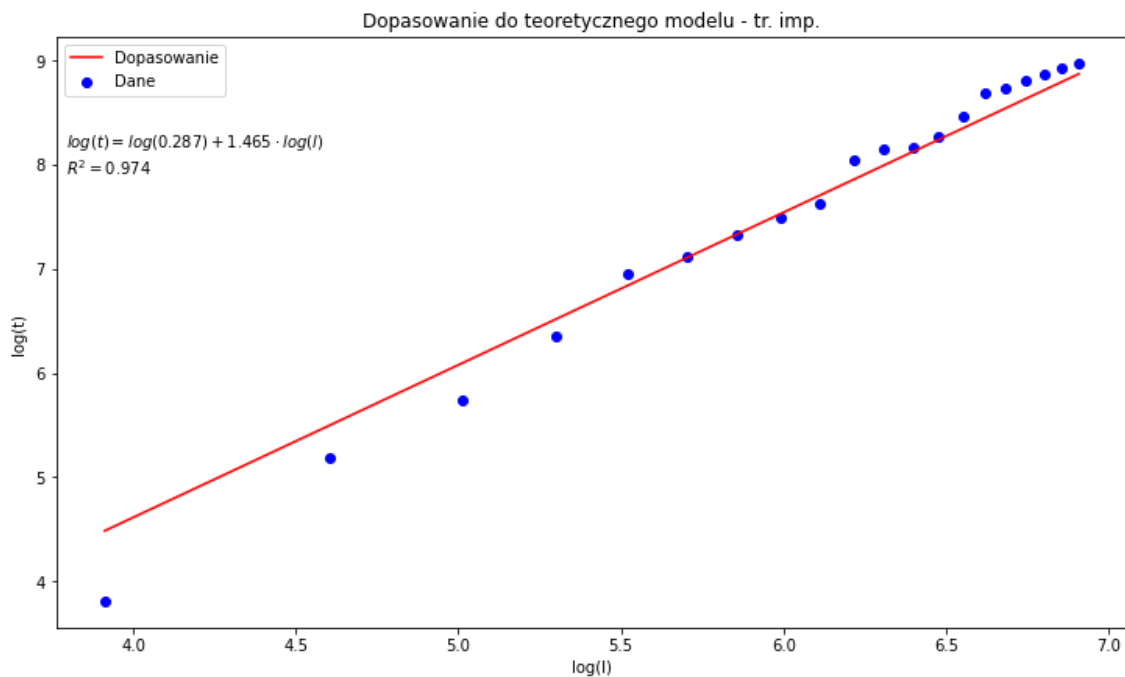
Rysunek 14: Dopasowanie modelu o teoretycznym asymptotycznym tempie wzrostu $n = 1,465$ do nieprzekształconych danych dla utworzonej implementacji.



Rysunek 15: Dopasowanie modelu o teoretycznym asymptotycznym tempie wzrostu $n = 1,465$ do przekształconych danych dla biblioteki GMP.



Rysunek 16: Dopasowanie modelu o teoretycznym asymptotycznym tempie wzrostu $n = 1,465$ do przekształconych danych dla języka Python.



Rysunek 17: Dopasowanie modelu o teoretycznym asymptotycznym tempie wzrostu $n = 1,465$ do przekształconych danych dla utworzonej implementacji.

Analizując tabelę 1. oraz powyższe wykresy, można zauważyć, że we wszystkich przypadkach wartości współczynnika determinacji są bliskie jedności. Jednak w niektórych z nich wartości błędów MSE oraz $RMSE$ są względnie duże. Co więcej, do dopasowania modelu do nieprzekształconych danych należy podejść z pewną ostrożnością, ponieważ w tym przypadku wykonywano obliczenia z wykorzystaniem metod mniej stabilnych numerycznie, co może mieć wpływ na wyznaczone wartości wykładnika n .

W poniższej tabeli przedstawiono błędy standardowe oraz błędy względne obliczonych współczynników a i n . Wykonując dopasowania do modelu o teoretycznej złożoności asymptotycznej, założono, że wartość n jest równa teoretycznej złożoności równej 1,465. Błędy standardowe oznaczono symbolem σ , a błędy względne symbolem δ .

Nazwa	a	σ_a	δ_a [%]	n	σ_n	δ_n [%]
a)	0,0056	0,0007	12,5	1,6120	0,0187	1,2
b)	0,0047	0,0044	93,6	1,3562	0,1411	10,4
c)	1765	0,0548	31,0	1,5524	0,0464	3,0
d)	0,0055	0,0003	5,5	1,6143	0,0079	0,5
e)	0,0083	0,0022	26,5	1,2627	0,0437	3,5
f)	0,0920	0,0109	11,8	1,6524	0,0195	1,2
g)	0,0023	0,0001	4,3	-	-	-
h)	0,0149	0,0001	0,7	-	-	-
i)	0,3165	0,0038	1,2	-	-	-
j)	0,0024	0,0001	4,2	-	-	-
k)	0,0136	0,0004	2,9	-	-	-
l)	0,2848	0,0106	3,7	-	-	-

Tabela 6: Wyznaczone wartości parametrów modelu i ich błędy.

Analizując powyższą tabelę można stwierdzić, że użycie przekształcenia logarytmicznego pozwoliło zmniejszyć błędy względne wyznaczonych współczynników, a co za tym idzie poprawić stabilność modelu.

Patrząc zarówno na współczynnik determinacji, błędy MSE i $RMSE$, jak i wartości błędów względnych wyznaczonych parametrów, najlepiej dopasowanymi modelami są modele teoretyczne dla przekształconych danych. Świadczy to o tym, że teoretyczne asymptotyczne tempo wzrostu dla algorytmu Toom-Cook-3 poprawnie opisuje badaną przez nas zależność.

7 Dodatek

7.1 Dokumentacja promptów

Do wygenerowania szkielethu kodu oraz implementacji algorytmów wykorzystano model językowy **Google Gemini 1.5 Pro**. Poniżej zestawiono prompty, które posłużyły do opracowania finalnego rozwiązania.

Prompt 1: Architektura i Szkielek Systemu

Cel: Stworzenie fundamentów struktury `WielkaLiczba` w języku C++ przy założeniu architektury.

Prompt Inicjujący

Zaprojektuj w C++ strukturę `WielkaLiczba` do obsługi arytmetyki o dowolnej precyzji (arbitrary-precision arithmetic).

- Użyj `std::vector<uint32_t>` jako wewnętrznego kontenera na słowa 32-bitowe.
- Przyjmij konwencję **big endian** (Most Significant Word pod indeksem zero).
- Dodaj pole `bool czy_ujemna`.
- Przygotuj szkielek dla algorytmu mnożenia **Toom-Cook 3 (Toom-3)** oraz niezbędne funkcje pomocnicze (porównywanie, wczytywanie z pliku, proste dodawanie i odejmowanie).

Prompt 2: Implementacja Logiki Algorytmu Toom-3

Cel: Odzworowanie logicznej struktury algorytmu, w tym rekurencji oraz operacji na współczynnikach.

Prompt Implementacyjny

Zaimplementuj algorytm mnożenia **Toom-Cook 3** dla klasy `WielkaLiczba`.

- Algorytm powinien dzielić liczbę na 3 części o długości $k = \lceil n/3 \rceil$.
- Zaimplementuj wyznaczanie punktów $v_0, v_1, v_{-1}, v_2, v_\infty$.
- Dodaj funkcję `div_exact` do dzieleni przez małe stałe (np. przez 2 i 6) występujących w fazie interpolacji.
- Pamiętaj o obsłudze znaków przy wyznaczaniu punktów ujemnych.
- W przypadku małych liczb, przełączaj się na algorytm Karatsuby lub mnożenie pisemne (progowe wartości N_0 i N_1).

Prompt 3: Optymalizacja i Refaktoryzacja

Cel: Eliminacja błędów wydajnościowych zidentyfikowanych podczas testów (nadmiarowe kopiowanie, nieefektywna reprezentacja big endian).

Prompt Optymalizacyjny

Mój obecny kod klasy `WielkaLiczba` wykazuje niską wydajność. Przeprowadź refaktoryzację w celu optymalizacji:

- **Zmiana reprezentacji:** Przejdź na format **little endian** (LSW first), aby zastąpić operację `insert(begin(), ...)` efektywnym `push_back`.
- **Eliminacja kopiowania:** Zmień przekazywanie parametrów z wartości na referencję do stałej na przykład: `(const WielkaLiczba&)` w celu uniknięcia kopiowania wektorów przy rekurencji.

7.2 Jak duże liczby można pomnożyć w ciągu sekundy?

W ramach rozszerzenia projektu podjęto próbę wyznaczenia granic wydajnościowych współczesnych rozwiązań w zakresie arytmetyki komputerowej. Głównym celem tej części badania było określenie rzędu wielkości liczb (wyrażonego w liczbie słów maszynowych), których operacja mnożenia mieści się w zadanym przybliżeniu czasowym: 1 milisekundy oraz 1 sekundy.

Do przeprowadzenia testów wykorzystano bibliotekę GNU MP (GMP), z racji najefektywniejszej implementacji. Należy zaznaczyć, że wyniki są ściśle związane z architekturą sprzętową komputera, na którym przeprowadzony jest eksperyment. Pomiary przeprowadzono na jednostce wyposażonej w procesor Intel Core i5-7400 3.0 GHz.

Na podstawie wstępnej empirycznej analizy ustalono, że:

- Granica czasu wykonania wynosząca 1 ms odpowiada liczbom o długości od 8 700 do 9 000 limbów (słów 32-bitowych).
- Granica czasu wykonania wynosząca 1 s odpowiada liczbom o długości od 3 650 000 do 3 700 000 limbów.

Generowanie danych testowych

W celu weryfikacji postawionych hipotez opracowano skrypt w języku Python, wykorzystujący mechanizmy opisane w rozdziale 7.3. Wygenerowano dwa zestawy danych testowych:

1. 7 par liczb o długości od 8 700 do 9 000 limbów z krokiem wynoszącym 50 słów.
2. 6 par liczb o długości od 3 650 000 do 3 700 000 limbów z krokiem wynoszącym 10 000 słów.

Wyniki eksperymentu

Dane poddano operacji mnożenia z wykorzystaniem implementacji opartej na bibliotece GMP (opisanej w rozdziale 7.3). Uzyskane wyniki prezentują się następująco:

Liczba słów	Czas wykonania	Jednostka
8 700	1,02190	ms
8 750	1,06987	ms
8 800	1,03674	ms
8 850	984,155	μs
8 900	972,745	μs
8 950	1,01806	ms
9 000	987,652	μs
3 650 000	977,098	ms
3 660 000	990,404	ms
3 670 000	1,04706	s
3 680 000	1,05475	s
3 690 000	1,04270	s
3 700 000	1,02102	s

Tabela 7: Wyniki pomiarów czasu mnożenia dla wybranych rzędów wielkości.

Aby zobrazować skalę problemu obliczeniowego, wyznaczono teoretyczną liczbę cyfr dziesiętnych dla rekordu o długości 3 660 000 słów maszynowych. Przyjmując, że każde słowo składa się z 32 bitów, całkowita liczba bitów B wynosi:

$$B = 3\,660\,000 \times 32 = 117\,120\,000 \text{ bitów}$$

Liczbę cyfr dziesiętnych D wyznacza się, korzystając z własności logarytmów. Dla liczby x , liczba jej cyfr w systemie o podstawie 10 jest równa $\lfloor \log_{10}(x) \rfloor + 1$. W przybliżeniu można przyjąć zależność:

$$D \approx B \times \log_{10}(2)$$

$$D \approx 117\,120\,000 \times 0,301029995$$

$$D \approx 35\,256\,633$$

Z powyższych obliczeń wynika, że badana liczba posiada ponad **35 milionów cyfr dziesiętnych**.

Przyjmując założenie o szerokości pojedynczej cyfry równej 0,5 cm, zapisanie tej wartości w jednej linii wymagałoby płaszczyzny o długości:

$$L = 35\,256\,633 \times 0,005 \text{ m} \approx 176\,283 \text{ m} \approx 176,3 \text{ km}$$

Tyle ile wynosi mniej więcej odległość między Gdańskiem a Toruniem.

Fakt, że biblioteka GMP jest w stanie wykonać operację mnożenia dwóch tak ogromnych obiektów w czasie poniżej jednej sekundy (990,404 ms), świadczy o niezwykle wysokiej efektywności zaimplementowanych algorytmów (prawdopodobnie Szybkiej Transformaty Fouriera - FFT) oraz współczesnych jednostek CPU.

7.3 Dokumentacja techniczna

Generowanie próbki

Próbka jest wygenerowana z użyciem języka Python, konkretnie za pomocą modułu **random** (patrz: *random* — *Generate pseudo-random numbers* [6]).

Zbiory są wygenerowane jak wspomniano w metodologii. Uruchomiony skrypt: **data_gen.py** tworzy 20 plików tekstowych, każdy przeznaczony liczbom o danej wielkości (ilości słów).

Każda z stu linii jest przeznaczona na jedną liczbę. Są one zapisane w bazie $\beta = 2^{32}$. Każde słowo maszynowe jest rozdzielone znakiem spacji i są one ułożone w kolejności od najbardziej znaczącej do najmniej znaczącej.

Skrypt generuje po kolei każde słowo za pomocą metody **.getrandbits(k)**, gdzie nasze $k = 32$. Metoda ta korzysta głównie z generatora Mersenne’a Twister’a, który generuje liczby pseudolosowe o rozkładzie jednostajnym (patrz: “Mersenne Twister: A 623-Dimensionally Equidistributed Uniform Pseudorandom Number Generator” [4]).

Implementacja w języku Python

Język Python posiada natywną obsługę liczb całkowitych o dowolnej precyzji (*arbitrary-precision integers*), co eliminuje konieczność stosowania zewnętrznych bibliotek. Rozmiar reprezentowanych liczb nie posiada sztywnego ograniczenia technicznego [5]; jedynym czynnikiem limitującym jest dostępna pamięć operacyjna komputera.

Skrypt **main_PYTHON.py** składa się z zestawu funkcji pomocniczych oraz funkcji głównej *main_func*, odpowiedzialnej za przeprowadzenie operacji mnożenia oraz pomiar czasu ich wykonania.

vector_to_int Funkcja pomocnicza służąca do konwersji wektora 32-bitowych słów maszynowych na typ **int** języka Python. Jako argument przyjmuje wektor słów, a zwraca obiekt o nieograniczonej precyzji. Proces rekonstrukcji liczby opiera się na przesunięciu bitowym w lewo (*bit-shift*) oraz sumowaniu kolejnych fragmentów danych przy użyciu operatora alternatywy bitowej (*OR*).

choose_time_unit Funkcja odpowiedzialna za automatyczny dobór jednostki czasu oraz odpowiednie skalowanie wartości liczbowej w celu poprawy czytelności wyników. Przyjmuje czas trwania operacji w mikrosekundach i zwraca krotkę (*wartość, jednostka*), gdzie wartość jest typu **float**, a jednostka typu **string**.

main_func Główna procedura sterująca procesem obliczeniowym. Realizuje ona mnożenie par liczb wczytanych z pliku, a następnie zapisuje czas egzekucji w pliku wynikowym. Funkcja przyjmuje dwie ścieżki do plików: wejściową oraz wyjściową.

Zgodnie z przyjętą metodologią, plik wejściowy musi zawierać parzystą liczbę rekordów (po jednym w każdym wierszu), reprezentujących liczby o podstawie $\beta = 2^{32}$. Funkcja wczytuje dane parami, dokonuje ich konwersji na typ **int**, a po wykonaniu mnożenia i sformatowaniu jednostek czasu, zapisuje wynik w formacie:

liczba słów maszynowych, czas (z precyzją do dwóch miejsc po przecinku), jednostka

Implementacja z wykorzystaniem biblioteki GNU MP

Biblioteka GNU MP (GMP) jest wysokowydajnym narzędziem napisanym w języku C, przeznaczonym do wykonywania operacji arytmetycznych na liczbach o dowolnej precyzji. W niniejszym projekcie wykorzystano wyłącznie moduł operacji na liczbach całkowitych (**mpz_t**). Wykorzystana implementacja pochodzi z oficjalnej dystrybucji i nie została poddana modyfikacjom.

vector_to_mmpz Funkcja pomocnicza dokonująca transformacji wektora typu **std::vector<uint32_t>** na obiekt struktury **mpz_t**. Jej zadaniem jest ułatwienie procesu importu danych i poprawa czytelności kodu źródłowego. Funkcja nie zwraca wartości (**void**), lecz modyfikuje przekazany przez referencję obiekt **mpz_t** za pomocą funkcji **mpz_import**, zgodnie ze specyfikacją dokumentacji GMP[3].

TimeResult W celu ustrukturyzowania danych pomiarowych zdefiniowano strukturę **TimeResult**, zawierającą dwa pola: **double value** (wartość liczbową) oraz **std::string unit** (oznaczenie jednostki). Ułatwia ona zarządzanie wynikami pomiarów i ich późniejszą prezentację.

choose_time_unit Funkcja analogiczna do wersji Pythonowskiej, dokonująca konwersji surowego pomiaru czasu (wyrażonego w nanosekundach) na obiekt struktury `TimeResult`.

main_func Główna funkcja odpowiedzialna za wczytywanie danych, realizację mnożenia oraz zapisanie wyników biblioteki GMP. Przyjmuje nazwy plików wejściowego i wyjściowego.

Podobnie jak w przypadku skryptu Python, wymagany jest plik z parzystą liczbą wierszy zawierających liczby o podstawie $\beta = 2^{32}$. Funkcja importuje dane do formatu `mpz_t`, wykonuje mnożenie, a następnie zapisuje parametry eksperymentu do pliku wyjściowego w formacie identycznym jak w przypadku implementacji w Pythonie.

Funkcja zwraca kod błędu 1 w przypadku niepowodzenia dostępu do plików lub 0 po pomyślnym zakończeniu wszystkich procedur obliczeniowych.

Analiza wyników

Zebrane dane zostały poddane analizie w celu zbadania zależności czasu potrzebnego na wykonanie mnożenia od rozmiaru mnożonych liczb oraz użytego narzędzia. Do uruchomienia wszystkich napisanych skryptów w odpowiedniej kolejności użyto narzędzia CMake. Za przygotowanie do analizy oraz samą analizę otrzymanych wyników odpowiada skrypt napisany w języku Python o nazwie `analysis.py`. Zawiera on funkcje opisane poniżej.

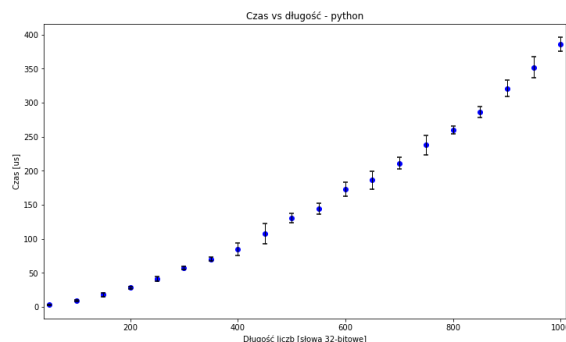
load_file Funkcja pobiera zawartość wybranego pliku ze zmierzonymi czasami potrzebnymi na wykonanie operacji mnożenia. Funkcja zwraca obiekt ramki danych zawierający dane przeznaczone do analizy. Funkcja ta zgłasza błąd w przypadku gdy szukany plik nie istnieje.

equalize_units Funkcja ujednolica jednostki, w których zapisane są czasy w wybranej ramce danych.

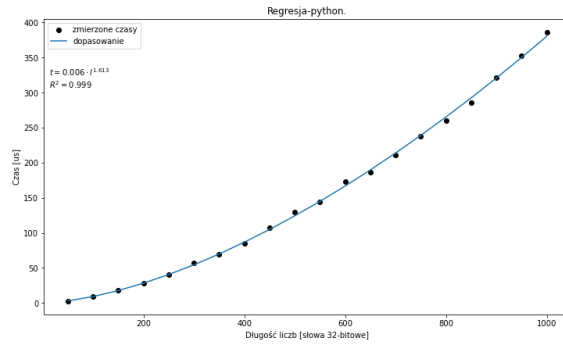
remove_outliers Funkcja ta usuwa wartości odstające w zbiorze czasów uzyskanych dla liczb o różnych długościach. Reguła, którą zastosowano do odrzucenia wartości odstających to tzw. reguła trzech σ , polegająca na odrzuceniu wartości różniących się o więcej niż trzy odchylenia standardowe od średniej.

calculate_stats Funkcja ta liczy medianę oraz odchylenie standardowe w zbiorze otrzymanych czasów dla liczb o zadanej długości.

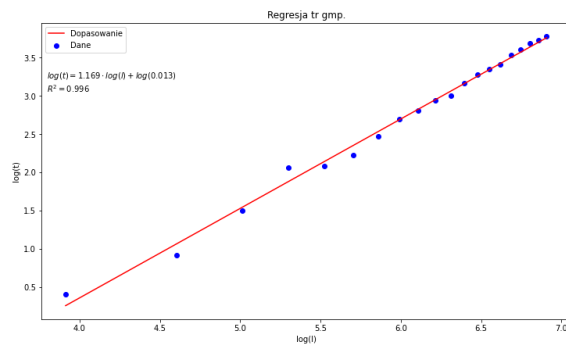
plot_one_method Funkcja ta generuje wykres dla wybranej metody. Oś pozioma przedstawia rozmiar liczb w słowach 32-bitowych, a oś pionowa czas w mikrosekundach. Średnie czasy przedstawione są za pomocą punktów. Odchylenia standardowe przedstawione są za pomocą punktów. Przykładowy wykres dla mnożenia wykonanego w Pythonie został przedstawiony poniżej.



draw_regression Funkcja ta dopasowuje regresję nieliniową o równaniu zdefiniowanym w funkcji `model` do uzyskanych czasów. W kolejnym kroku generowany jest odpowiedni wykres. Przykładowy wykres dla mnożenia wykonanego w Pythonie został przedstawiony poniżej, tutaj dla przypadku gdy dopasowywana funkcja jest funkcją liniową.



draw_transformed_regression Funkcja ta wykonuje przekształcenie logarytmiczne dla uzyskanych czasów i długości liczb. Następnie do przekształconych danych dopasowuje model dany równaniem $\log y = n \cdot \log x + \log a$. W wyniku wywołania funkcji rysowany jest odpowiedni wykres oraz zwracane są informacje o dopasowaniu modelu do danych. Przykładowy wykres dla mnożenia w bibliotece GMP został przedstawiony poniżej.



workflow Funkcja ta odpowiada za wywołanie funkcji przedstawionych powyżej w odpowiedniej kolejności. Dodatkowo generowane są dwa raporty z analizy danych. Plik *Raport.csv* zawiera informacje o dopasowaniach wszystkich modeli. Plik *Raport_1.csv* zawiera wszystkie obliczone mediany czasów oraz odchylenia standardowe.

References

- [1] Autorzy Wikipedii. *GNU Multiple Precision Arithmetic Library* — *Wikipedia, The Free Encyclopedia*. https://en.wikipedia.org/wiki/GNU_Multiple_Precision_Arithmetic_Library. [Dostęp: 27.04.2026]. 2026.
- [2] Richard P. Brent and Paul Zimmermann. *Modern Computer Arithmetic*. Vol. 18. Cambridge Monographs on Applied and Computational Mathematics. Cambridge: Cambridge University Press, 2010. ISBN: 978-0-521-19469-3.
- [3] Torbjörn Granlund and the GMP development team. *GNU MP: The GNU Multiple Precision Arithmetic Library*. 6.3.0. Dostęp online. 2023. URL: <https://gmplib.org/manual/>.
- [4] Makoto Matsumoto and Takuji Nishimura. “Mersenne Twister: A 623-Dimensionally Equidistributed Uniform Pseudorandom Number Generator”. In: *ACM Transactions on Modeling and Computer Simulation* 8.1 (1998), pp. 3–30. DOI: 10.1145/272991.272995.
- [5] Python Software Foundation. *Integer Objects*. Online; accessed 2026. Python Software Foundation. URL: <https://docs.python.org/3/c-api/long.html>.
- [6] Python Software Foundation. *random* — *Generate pseudo-random numbers*. Online; accessed 2026. Python Software Foundation. 2024. URL: <https://docs.python.org/3/library/random.html>.